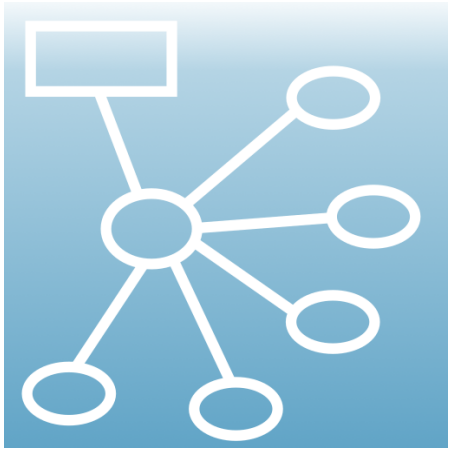

ThinkComposer User Manual

Version 1.5.1619 - Visual thinking, domain modeling, interchange, and output generation

Instrumind ThinkComposer / maintained fork by tbm0115

2026-06-19

Contents

1 Overview

ThinkComposer is a visual thinking tool for understanding problems, designing solutions, and expressing knowledge. It helps users create concept maps, mind maps, models, diagrams, and structured visual documents with detailed content attached to the visual elements.

This manual documents the independently maintained `tbm0115/ThinkComposer` fork while preserving the original ThinkComposer product model, terminology, and attribution.

The product is intended for professionals, academic users, students, analysts, designers, managers, and teams who need graphic means to explore, organize, and communicate knowledge. It is especially useful when the work involves discovery, problem analysis, solution design, research, process modeling, or knowledge transfer.

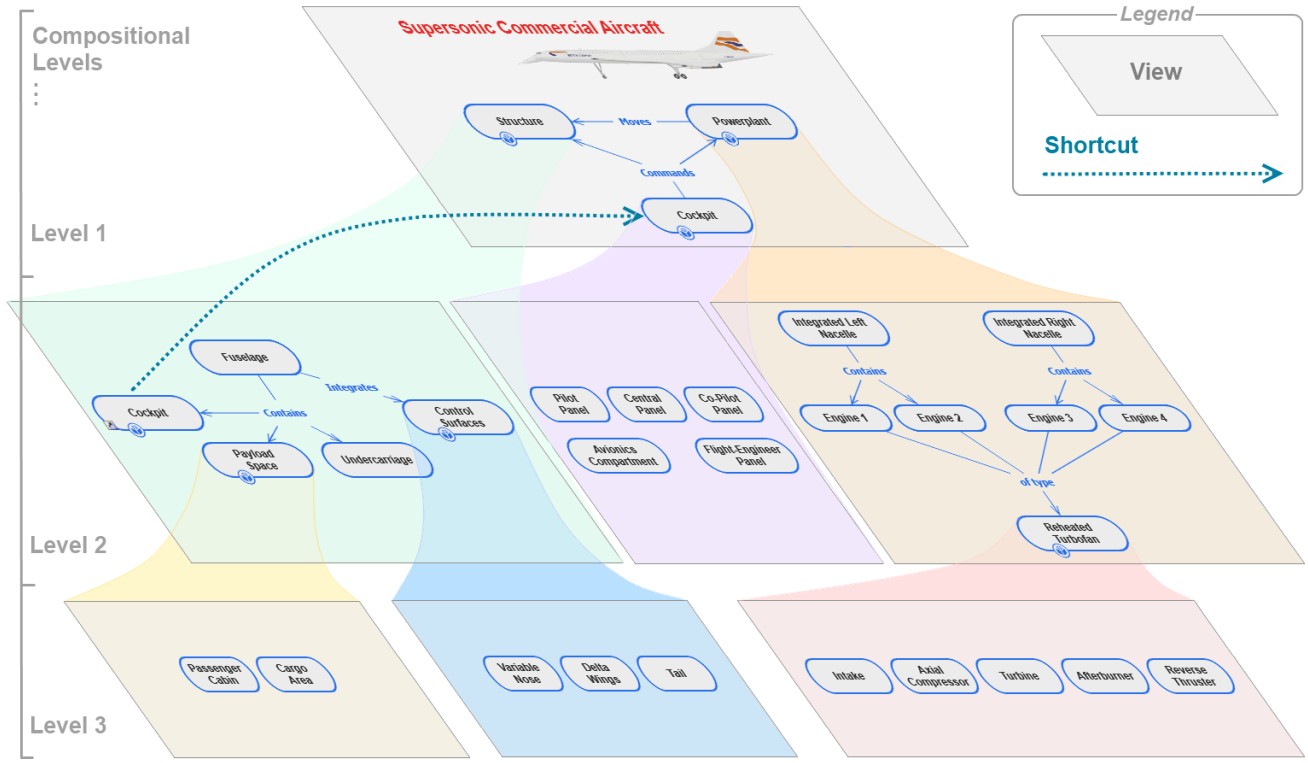

Figure 1.1: Composition example

1.1 Vision

ThinkComposer combines visual mapping with a typed information model. A user can start with a free-form diagram, then progressively add meaning through domains, definitions, details, markers, relationships, and generated outputs.

The guiding ideas are:

- Visual thinking should stay expressive enough for early exploration.
- Models should become precise when precision matters.
- Diagrams should support details, attachments, data tables, and generated files, not just shapes and lines.
- Domain definitions should let different fields use their own concepts, relationship types, markers, formats, and tables.
- Generated output should be derived from the model through templates instead of being manually recreated.

1.2 Product Context

ThinkComposer sits between ordinary diagramming, mind mapping, modeling, and documentation tools.

It can be used for:

- conceptual maps and mind maps
- business and system diagrams
- analysis and design models
- domain-specific notation
- knowledge bases with attached details
- structured reports
- code or text generation from model data
- JSON-backed native persistence and JSON-assisted interchange with external tools and AI workflows

The application remains a native Windows desktop tool built with Windows Presentation Foundation. Modern `.tcom` and `.tdom` files are package containers whose authoritative saved payloads are JSON. The explicit JSON import/export commands remain editable exchange workflows layered on top of normal Open/Save.

1.3 How ThinkComposer Thinks About Work

ThinkComposer separates user work into two main document types:

- A **Composition** contains knowledge about a specific subject, project, system, process, or area of work.
- A **Domain** defines the concepts, relationships, tables, markers, visual formats, and output templates available to compositions in a field.

Multiple compositions can use the same domain. A composition also carries an embedded domain snapshot so it can remain self-contained.

1.4 Working Style

A typical workflow is:

1. Create or open a composition.
2. Choose a domain that fits the work.
3. Add concepts and relationships to a diagram view.
4. Attach details, tables, links, images, notes, or markers where needed.
5. Use layout and appearance tools to make the view readable.
6. Export reports, images, PDF, JSON, or generated text files.
7. Update the domain or composition through JSON when structured external editing is useful.

The rest of this manual explains each layer in that workflow, from the base model to current import/export and generation features.

2 Base Model

ThinkComposer's base model is the vocabulary shared by every composition and domain. It explains what the user creates, how visual objects represent meaning, and how domains constrain or enrich that meaning.

2.1 Working Documents

ThinkComposer uses two main document types.

Document	File type	Purpose
Composition	`.tcom`	A self-contained knowledge document containing ideas, views, details, visuals, and an embedded domain snapshot.
Domain	`.tdom`	A reusable definition package containing concept types, relationship types, marker definitions, table structures, formats, base content, and output templates.

A composition can be based on an external domain, but the composition file also stores enough domain information to remain portable.

2.2 Common Object Properties

Most user-visible model objects share these properties:

Property	Meaning
`Name`	Human-readable title.
`TechName`	Stable technical identifier, suitable for generated output and matching.
`Summary`	Short descriptive text.
`Description`	Richer explanatory text. JSON interchange exports this as plain text.
`TechSpec`	Technical specification text, often used by templates, code generation, or AI-assisted workflows.
`Version`	Optional version metadata such as creator, dates, annotation, and version number.

Use `Name` for people and `TechName` for tools. A good `TechName` is stable, unique in its context, and free

of display-only wording.

2.3 Compositions

A composition is the user's main working document. It owns the root idea, the available views, the active view, model content, visual representations, details, and embedded domain.

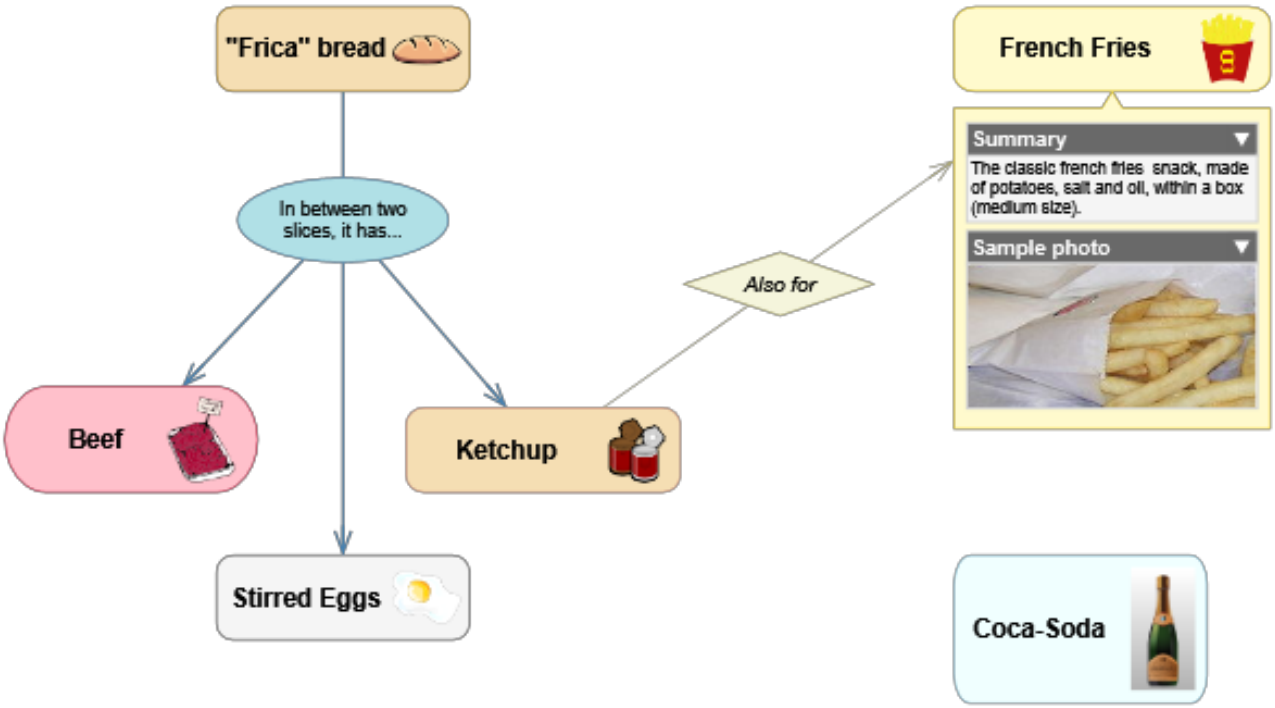

Figure 2.1: Composition and domain relationship

The root composition can contain nested ideas. Concepts can themselves be composite, giving a composition multiple levels of detail and multiple diagram views.

2.4 Views

A view is a diagram canvas showing the composite content of a composition or idea. The same idea can appear in more than one view through visual representations or shortcuts.

Views store display-related settings such as:

- background
- grid size and snap-to-grid
- page display scale
- visible labels and indicators
- concept and relationship symbols
- complements such as notes, callouts, legends, and group regions

2.5 Ideas

An idea is the base semantic item in a composition. There are two main kinds:

- **Concepts** are objects, actors, states, activities, data entities, or other things being described.

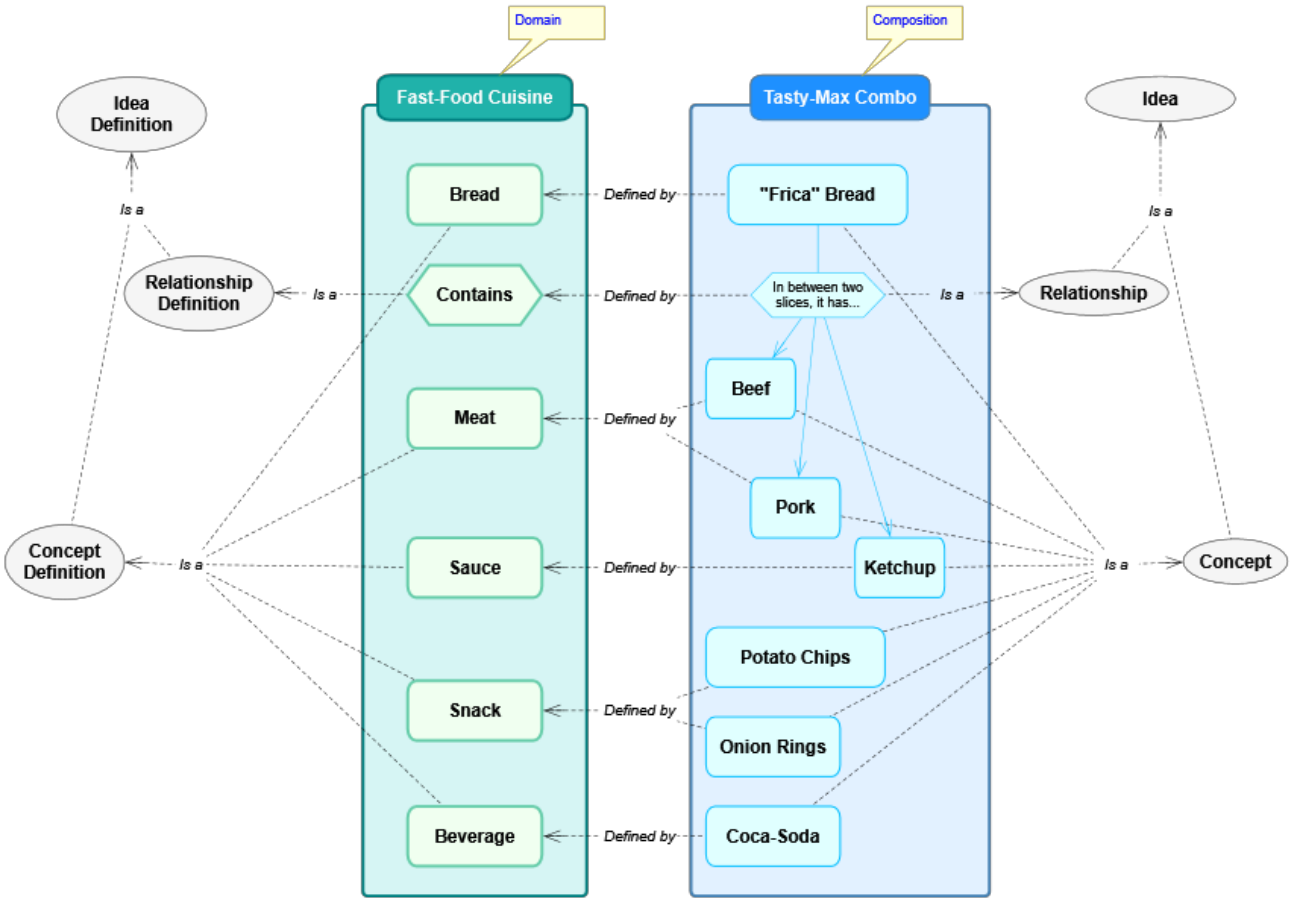

Figure 2.2: Nested composition views

- **Relationships** connect concepts through roles and give those links meaning.

Ideas can carry details, custom fields, markers, version information, and visual representations. An idea can also be composite, meaning it contains child ideas and one or more views.

2.6 Symbols And Visual Representations

A symbol is the visible body of a concept or the central symbol of a relationship. A visual representation groups the symbol and related visual elements that show an idea in a view.

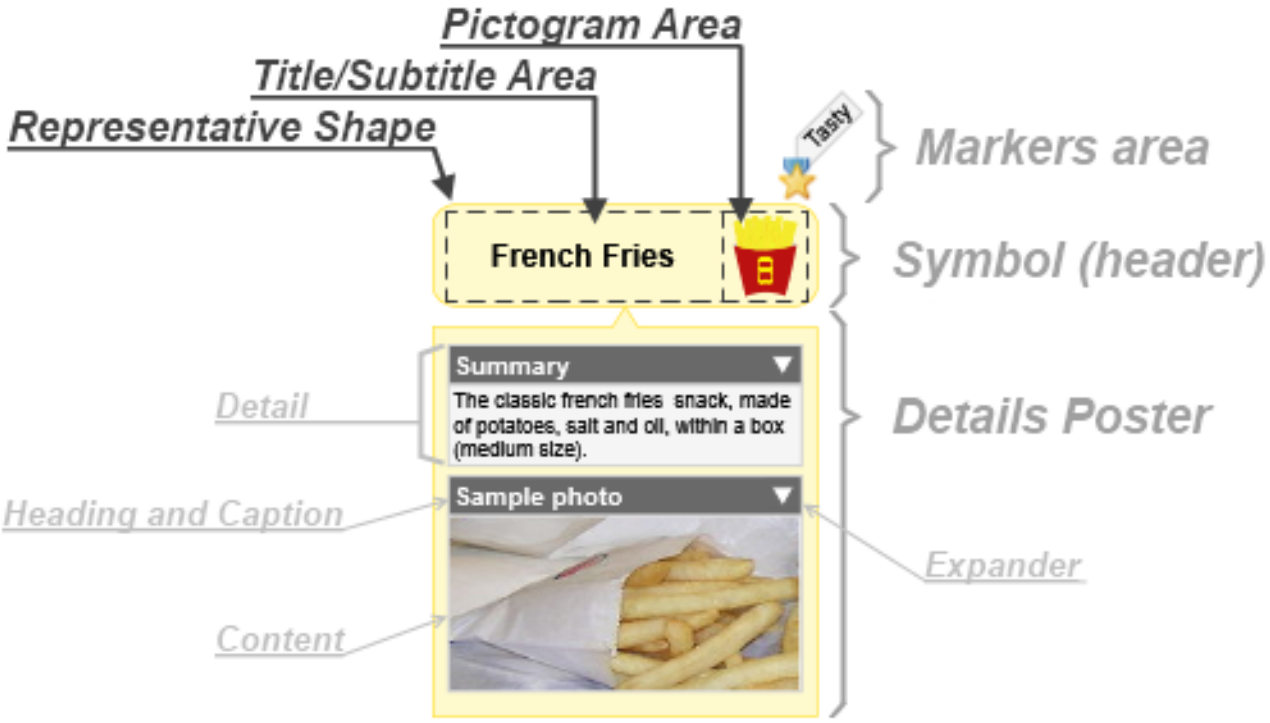

Figure 2.3: Symbol parts

Symbols can show:

- title and subtitle text
- pictograms
- markers
- a details poster
- composite content as a mini-view
- connectors for relationships

Visual representations are separate from semantic ideas. This is why the same idea can have a primary visual in one view and a shortcut visual somewhere else.

2.7 Shortcuts

A shortcut is a visual representation of an existing idea. It is not a duplicate concept or relationship. Use shortcuts when one semantic idea needs to appear in another location or context.

Current JSON interchange preserves shortcuts with `isShortcut: true`, so round-tripping does not accidentally duplicate semantic ideas.

2.8 Details

Details attach rich content to ideas. They let a diagram element carry structured and unstructured information without crowding the diagram itself.

Figure 2.4: Details poster

Supported detail kinds include:

- attachments
- resource links
- internal links
- tables
- custom fields
- text-like details such as descriptions and technical specifications

2.9 Attachments And Links

Attachments embed content obtained from an external source, such as an image or data file. Resource links reference external resources such as files, folders, or web addresses. Internal links reference properties or related internal objects.

Attachments and links remain native `.tcom` content. JSON interchange exports safe metadata and text where supported, but it does not inline large binary payloads.

2.10 Tables And Custom Fields

Tables store structured records inside ideas or domains. A table definition declares fields, label fields, required fields, contained-table fields, and display behavior.

Custom fields are table-record values attached to a specific idea. Output templates can access them through the _ alias, for example:

```
Also known as: {{ _['Alias'] }}
```

2.11 Concepts

A concept is a concrete idea. Its behavior and visual defaults come from a Concept Definition in the active domain.

Concepts may be:

- atomic or composite
- versionable
- decorated with markers
- visually represented by different symbol formats
- assigned custom fields and details
- related to other concepts through relationships

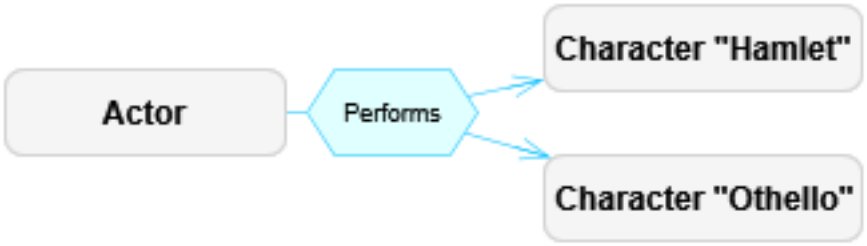

Figure 2.5: Concept examples

2.12 Relationships

A relationship is an idea that links other ideas through role-based links. A relationship may have a central symbol and visible connectors, or it may hide its central symbol when the relationship is simple.

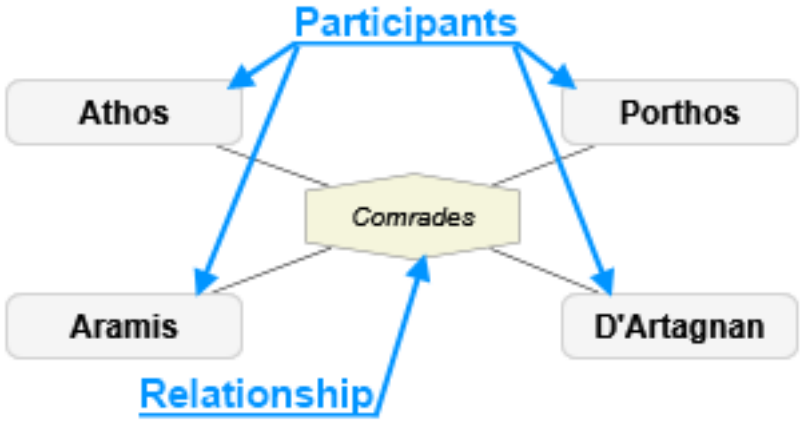

Figure 2.6: Relationship examples

Relationships support:

- directional or non-directional semantics
- origin/source and target/destination roles
- participant roles for non-directional relationships
- link role variants such as multiplicity/cardinality
- optional link descriptors
- custom visual connector formats

2.13 Connectors And Link Roles

Connectors are the visual lines between symbols. A connector represents a `RoleBasedLink`, which joins a relationship to an associated idea through a Link Role Definition.

Figure 2.7: Connector examples

Link role definitions can constrain:

- which idea definitions are linkable
- maximum number of connections
- whether related ideas are ordered
- which role variants are allowed
- plug styles and connector appearance

2.14 Markers

Markers are small visual annotations assigned to ideas. A marker can also have an optional descriptor. They are useful for priority, status, ownership, review state, classification, or any domain-specific tagging.

2.15 Complements

Complements are visual objects attached to the view or to symbols. They enrich diagrams without becoming semantic concepts or relationships.

Complement	Purpose
Legend	Explains visual conventions used in the view.
Info-Card	Displays selected object metadata.
Image	Adds a visual image to the diagram.
Text	Adds independent text.
Stamp	Adds status-like visual text.
Note	Adds explanatory annotation.
Callout	Adds a note with a pointer to a symbol.
Quote	Adds a callout styled as quoted text.

Complement	Purpose
Group Region	Adds a background grouping boundary attached to a target idea.
Group Line	Adds a line-like grouping/lifeline complement.

2.16 Domains

A domain defines the meaning and visual language available to compositions. It is the metamodel for a business area, discipline, or notation.

A domain may define:

- concept definitions
- relationship definitions
- link role definitions
- marker definitions
- table structures
- base tables
- external languages
- brushes and text formats
- symbol and connector formats
- detail designators
- output templates

2.17 Idea Definitions

Idea Definitions are shared ancestors for Concept Definitions and Relationship Definitions. They declare the structural, visual, and information features of ideas created from them.

Important definition settings include:

- whether ideas are composable
- whether ideas are versionable
- allowed composite-content domain
- custom fields table definition
- detail designators
- visual symbol shape and format
- automatic creation behavior
- grouping behavior through Group Regions or Group Lines

2.18 Brushes, Text Formats, And Symbol Formats

Domains define reusable appearance pieces so diagrams remain consistent.

Brushes control fills and strokes. Text formats control title, subtitle, detail heading, content, and caption text. Symbol formats combine shapes, placement, pictograms, title areas, details posters, background brushes, line brushes, and flip/tilt behavior.

2.19 Detail Definitions

A detail designator declares which detail kind an idea or definition may contain. Detail definitions can attach tables, attachments, links, and custom detail content to ideas in a controlled way.

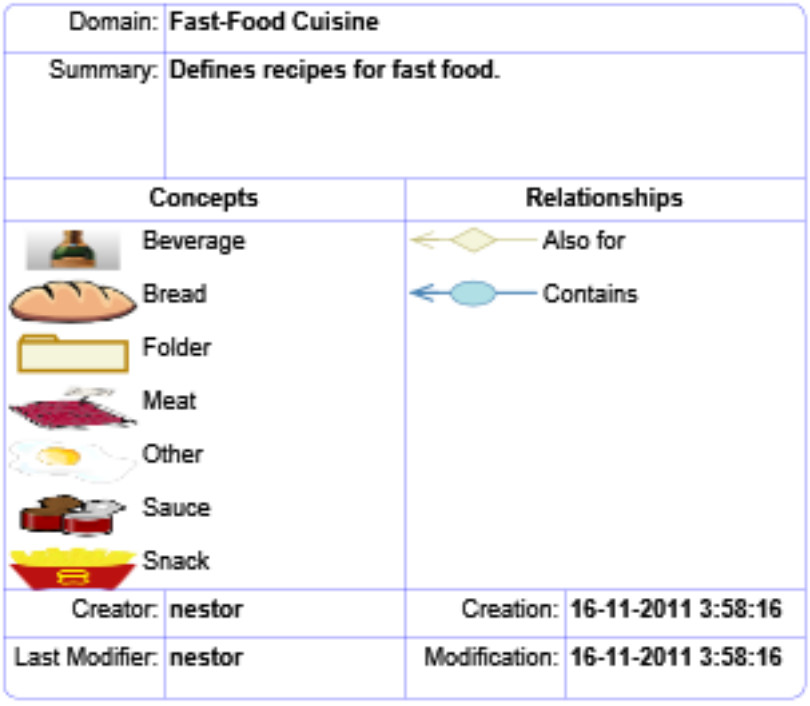

Figure 2.8: Marker examples

2.20 Output Templates

Output Templates generate text files from compositions, concepts, and relationships. They use Liquid-style markup plus ThinkComposer control markup and helper filters.

Domains can define:

- composition-level templates
- base concept templates
- base relationship templates
- definition-specific templates
- templates per external language
- subtemplates for reusable fragments

Current output-template behavior is described in Current Features and Template Language.

2.21 Concept And Relationship Definitions

Concept Definitions describe concept types. Relationship Definitions describe relationship types, directionality, central-symbol behavior, and role definitions.

Relationship Definitions may hide their central symbol when simple, show a name label when hidden, and restrict valid source/target definitions through role compatibility.

2.22 Marker Definitions, Table Structures, And Base Tables

Marker Definitions define the icons and descriptors available for marking ideas.

Table Structure Definitions define reusable table schemas. Base Tables store domain-level reference data, such as units, states, options, or other lookup records used by templates and details.

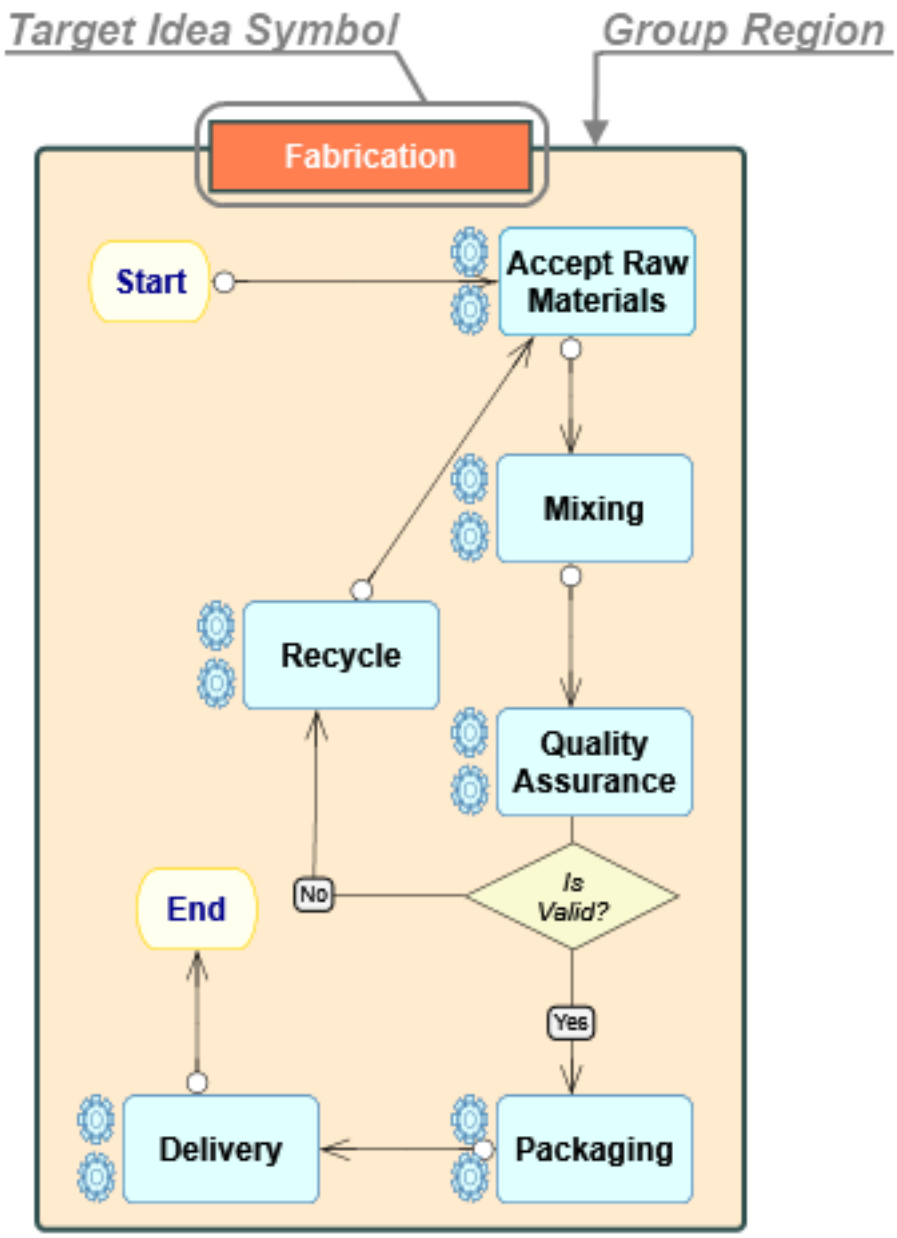

Figure 2.9: Complement examples

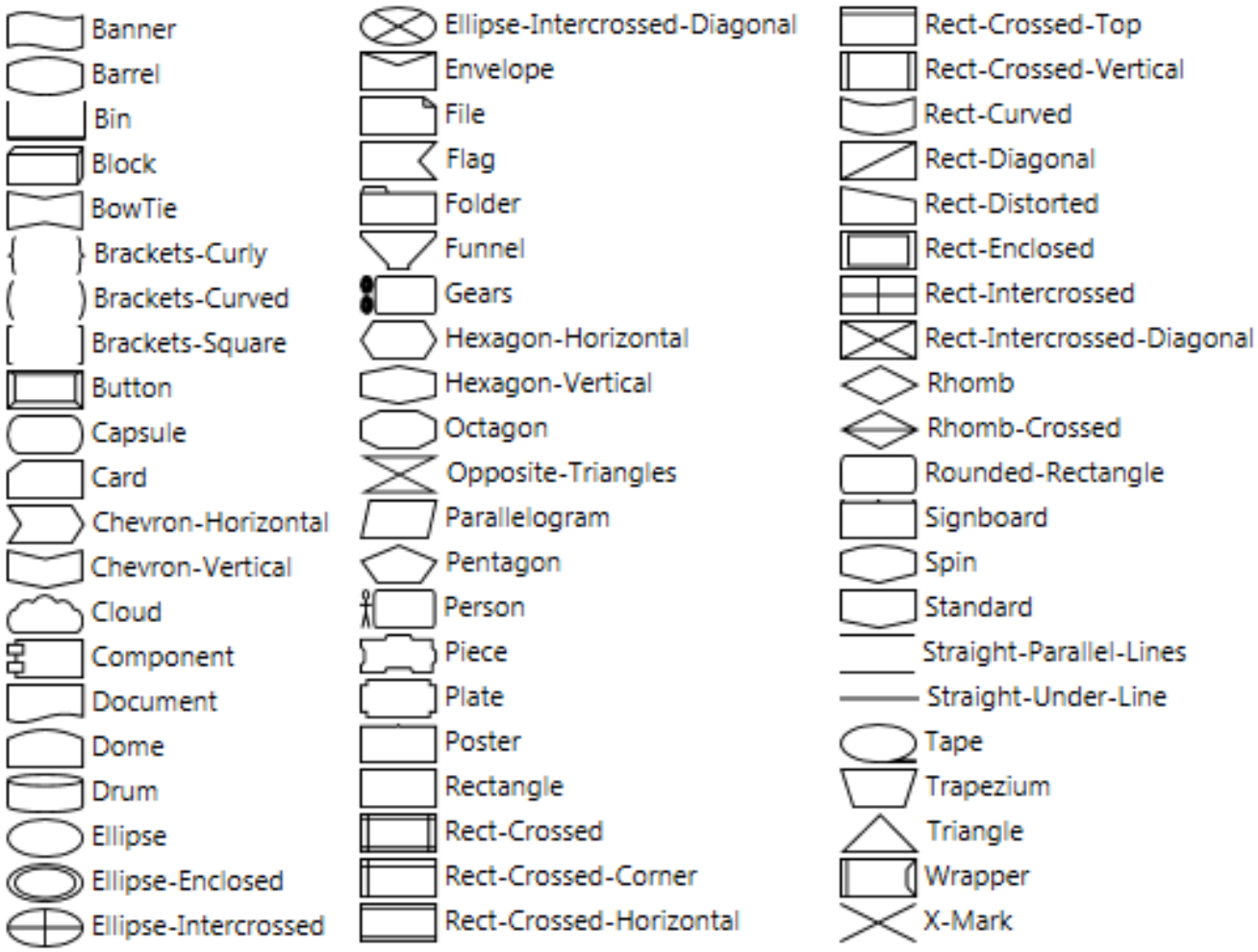

Figure 2.10: Domain definitions overview

Figure 2.11: Symbol format parts

Figure 2.12: Shape and brush examples

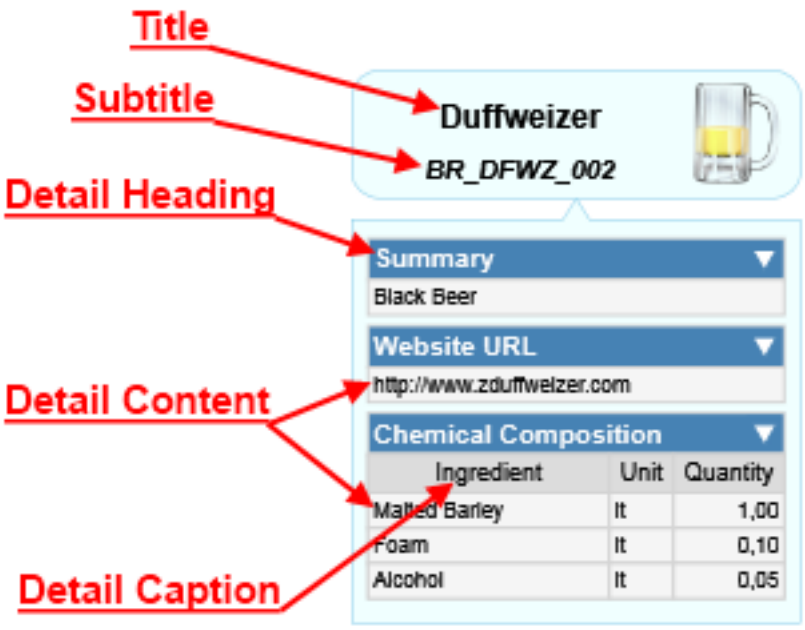

Figure 2.13: Details format example

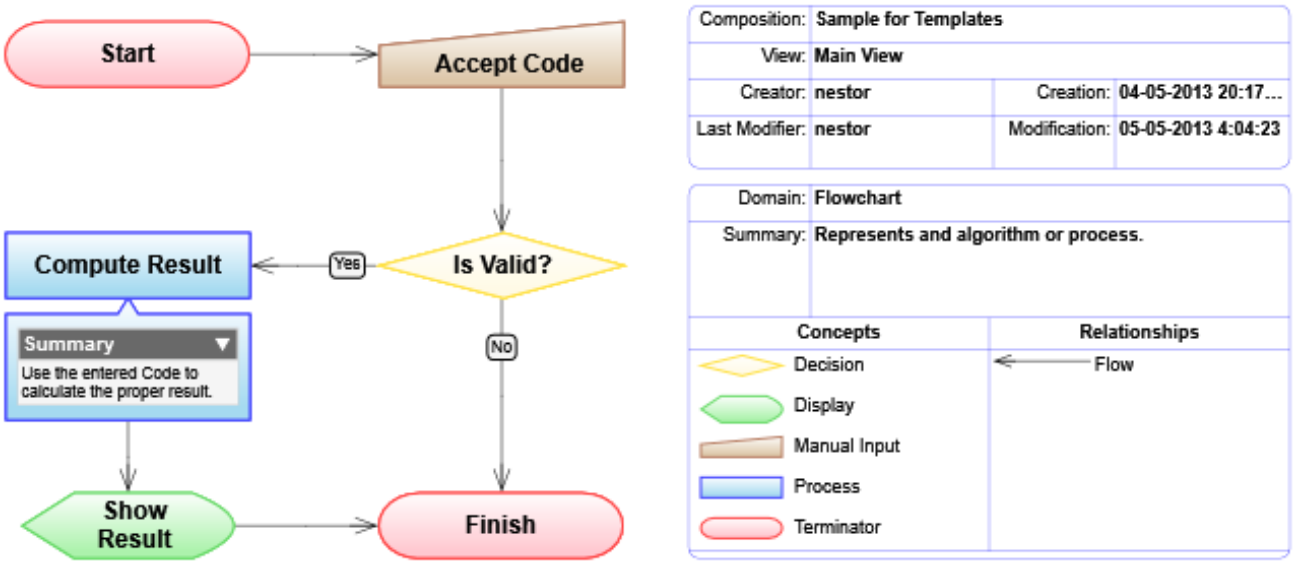

Figure 2.14: Output template inheritance

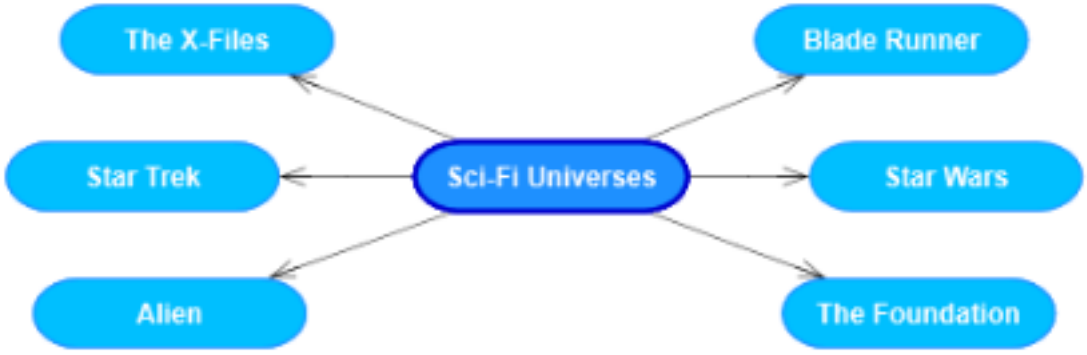

Figure 2.15: Concept definition examples

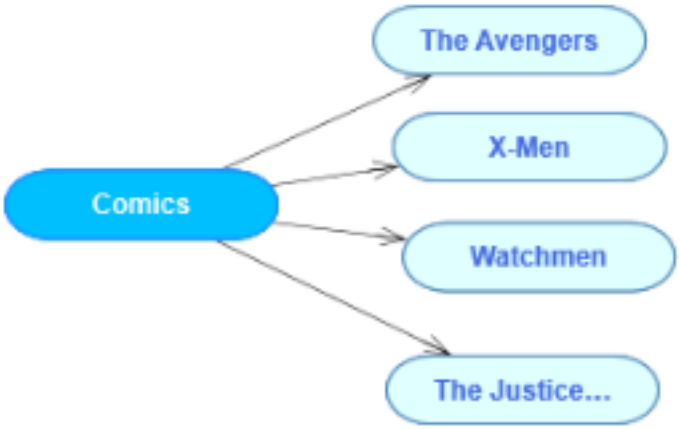

Figure 2.16: Relationship definition examples

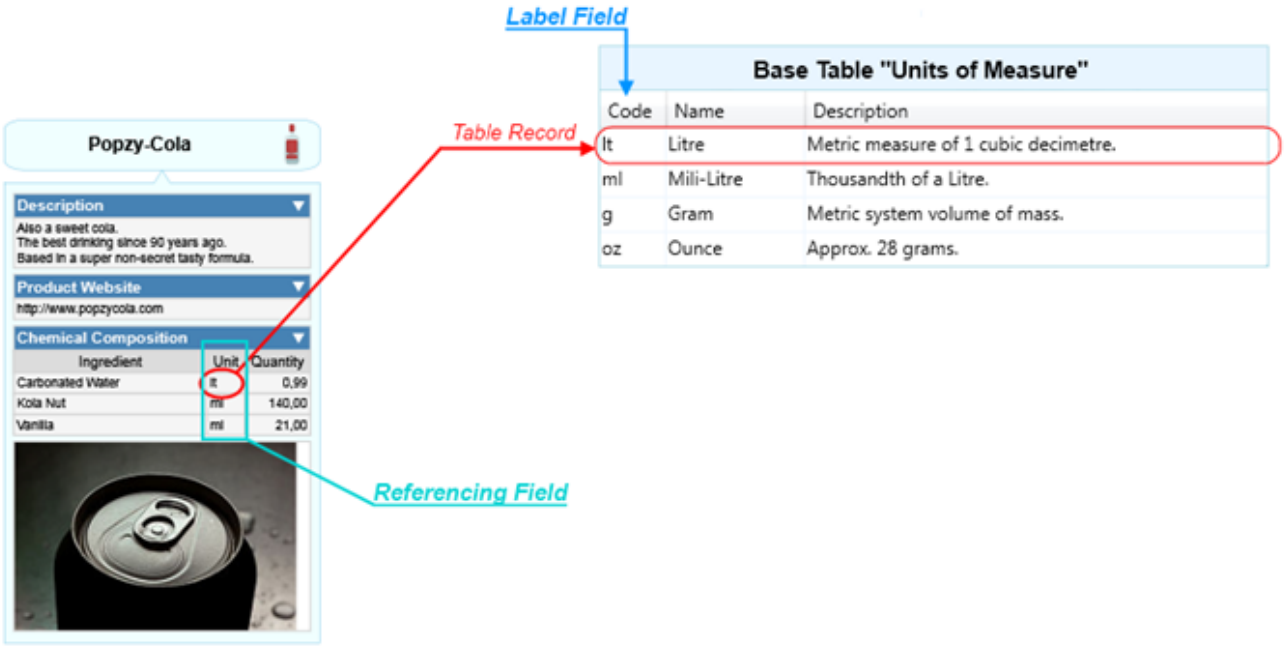

Figure 2.17: Table structure example

2.23 External Languages

External Languages identify output targets such as XML, JSON, Markdown, code, Mermaid, or other text languages. Output templates are grouped by external language so the same model can generate different kinds of output.

3 Application Guide

This guide covers day-to-day use of ThinkComposer: setup, the main window, editing diagrams, working with composition content, and producing reports.

3.1 Setup

3.1.1 Requirements

ThinkComposer is a Windows desktop application. It is designed for modern desktop PCs and advanced laptops with enough screen area for visual modeling. It uses Windows Presentation Foundation for high-definition diagram rendering.

3.1.2 Install And Uninstall

Install ThinkComposer through the provided installer. The installer includes the application, predefined domains, runtime assets, and the local PDF manual artifact used by older releases.

The installer also includes the headless command-line interface and PATH helper described in Command-Line Interface.

Use Windows application management or the installed uninstaller to remove the product.

3.1.3 Version Update

The release notes in this repository identify the current maintained fork build as `1.5.1619`. Older manuals may still show document version `1.5.13.1127`; this Markdown manual is the maintained source intended to replace that static PDF.

Before updating a production installation, save or copy important `.tcom` and `.tdom` files.

3.1.4 License Activation

Legacy builds include license activation behavior. Follow the installed application prompts for activation, trial control, or license updates.

3.2 Main Window

The main window presents the active project, a menu toolbar, palettes, a diagram workspace, properties, and application messages.

Important areas include:

Area	Purpose
Menu Toolbar	Global project commands and composition editing commands.

Area	Purpose
Project controls	Open, save, import, export, report, and domain-related commands.
Compose controls	Editing commands for concepts, relationships, details, appearance, and view behavior.
Palettes	Available concept definitions, relationship definitions, markers, and other domain objects.
Workspace	The active diagram view.
Status/log area	Operational messages, import/export diagnostics, and warnings.

Current import, layout, and output-generation commands log detailed diagnostics to the lower-left application log. Dialogs intentionally stay concise.

3.3 Working With Compositions

Create a composition when you need a new knowledge document. Choose an appropriate domain at creation time. The domain determines what concept and relationship definitions are available, how symbols look, and what details or templates can be used.

Save the native `.tcom` package as the source of truth. Modern packages persist their authoritative content in root JSON payloads; manual JSON exports, reports, PDFs, images, and generated files are exchange or publication artifacts.

3.4 Working With Diagram Views

A diagram view presents the composite content of the composition or a composite idea. You can open nested views, show composite content as details, and use shortcuts to show the same semantic idea in more than one place.

3.5 Editing Symbols

Symbols can be selected, moved, resized, formatted, sent forward/backward, and connected. Many symbol behaviors are inherited from the domain definition, but individual visuals can still be manually arranged.

Use the Appearance commands for larger cleanup:

- `Fit Concept Width to Text`
- `Route Links with Obstacle Avoidance`
- `Arrange as Spider Map`
- `Arrange as Hierarchy Map`
- `Arrange as Flowchart`
- `Arrange as System Map`

These commands are covered in Current Features.

3.6 Creating Concepts

Create concepts by using the concept palette, context menus, shortcuts, or automatic creation behavior defined by the domain.

When a concept is created:

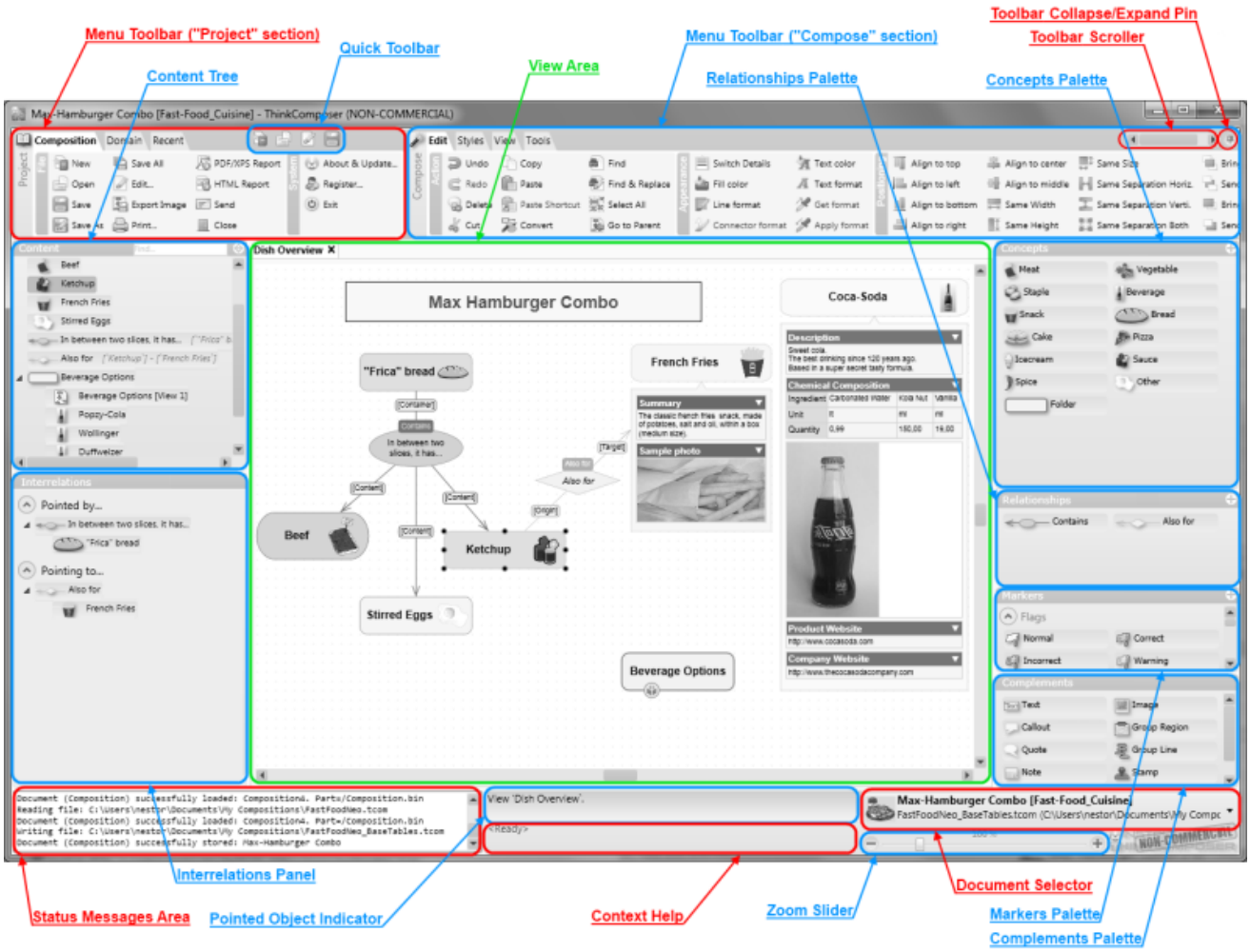

Figure 3.1: Main window

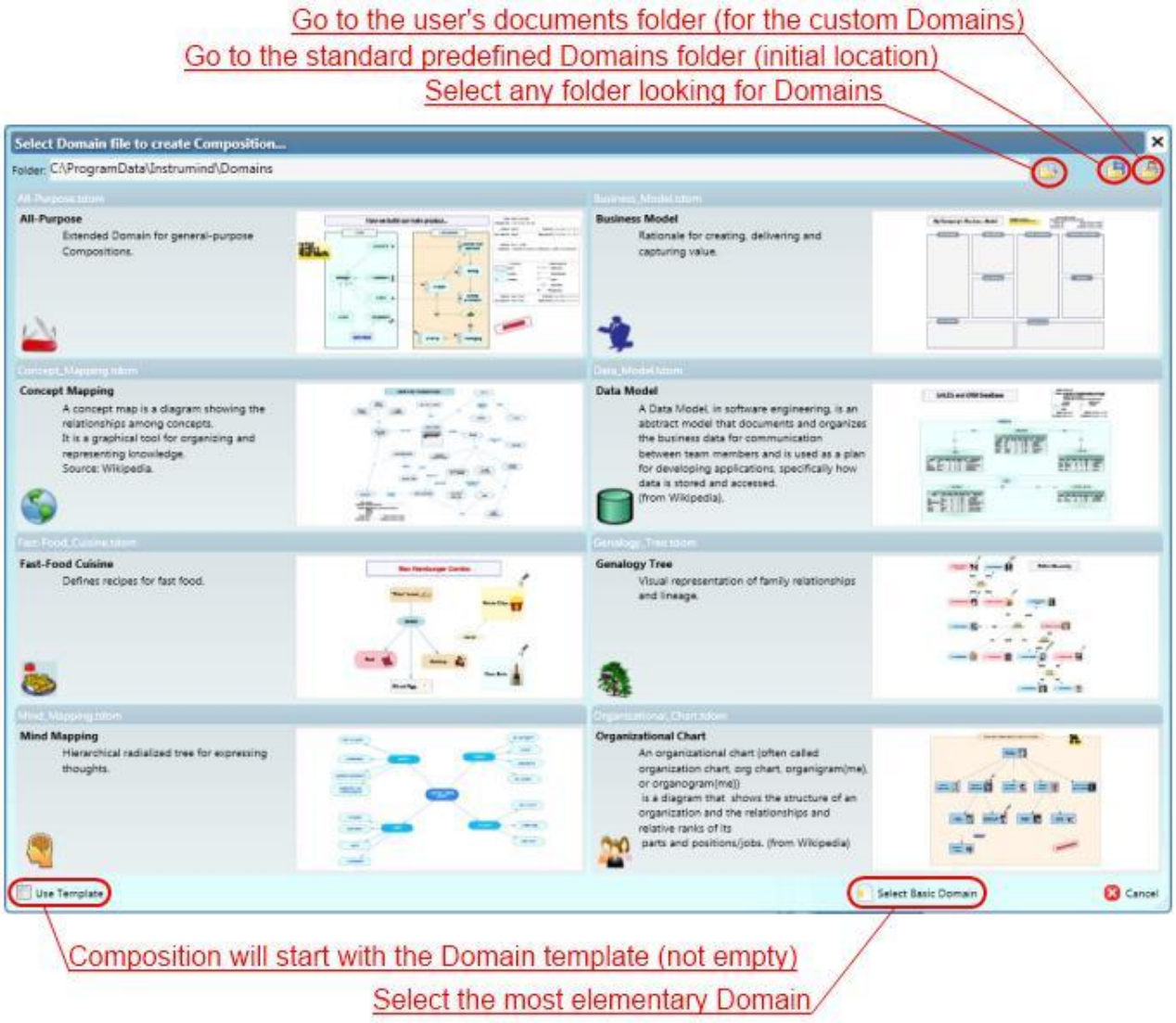

Figure 3.2: Composition workspace

1. Its Concept Definition supplies the semantic type.
2. The domain supplies visual defaults.
3. The active view receives a visual representation unless creation is model-only.
4. Details, custom fields, markers, and composite behavior become available according to the definition.

3.7 Creating Relationships

Create relationships by choosing a relationship definition and connecting concepts through the allowed roles. For directional relationships, ThinkComposer distinguishes origin/source and target/destination roles.

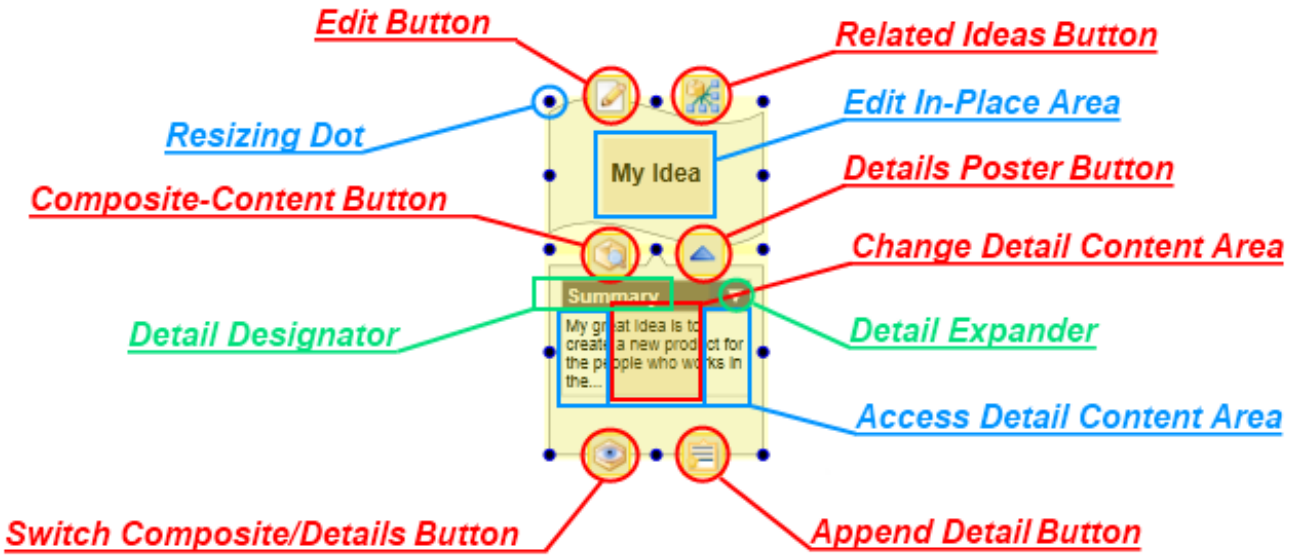

Figure 3.3: Relationship editing

Relationship compatibility depends on the active domain. If a relationship definition restricts allowed endpoint definitions, invalid links are rejected or reported during import.

3.8 Extending Or Modifying Relationships

Relationships can be extended by adding or adjusting role-based links when the relationship definition allows it. Link descriptors and role variants can annotate a connector without changing the relationship's own name or summary.

For generated or JSON-imported diagrams, prefer endpoint-corridor relationship placement so visible relationship centers stay near the concepts they connect.

3.9 Converting Ideas

When domain rules allow it, ideas can be converted or redefined so the model remains meaningful while the visual organization evolves.

Use conversion carefully in mature compositions because templates, relationship compatibility, details, and domain-specific semantics may depend on definitions.

3.10 Assigning Markers

Markers communicate status, priority, classification, or other domain-specific annotations.

Assign markers through the marker palette or context commands. Marker visibility can be controlled at the view level.

3.11 Creating Complements

Complements add visual context around ideas and views. Use them for explanatory notes, callouts, images, legends, info-cards, and grouping boundaries.

Figure 3.4: Complement in a view

Group Regions are especially useful for system boundaries, subgroups, and visual containers. Current System Map layout can create or update a visible Group Region around detected internal components.

3.12 Creating Shortcuts

Use **Replace with Shortcut…** on a non-shortcut concept symbol when you want the selected visual to point to an existing idea instead of representing its original idea. On a shortcut symbol, use **Go to Original** to navigate to a primary visual representation.

Shortcuts preserve one semantic identity across multiple views or locations.

3.13 Selection, Pan, And Zoom

Use selection to scope editing and layout commands. Many current Appearance commands operate on selected concept symbols or selected connectors when available, and otherwise prompt before acting on all visible objects.

Pan and zoom are view navigation tools only. They do not change the model.

3.14 Reporting

ThinkComposer can generate reports from a composition and export views as images or PDFs.

Composition reports are publication artifacts. The `.tcom` file remains the authoritative source.

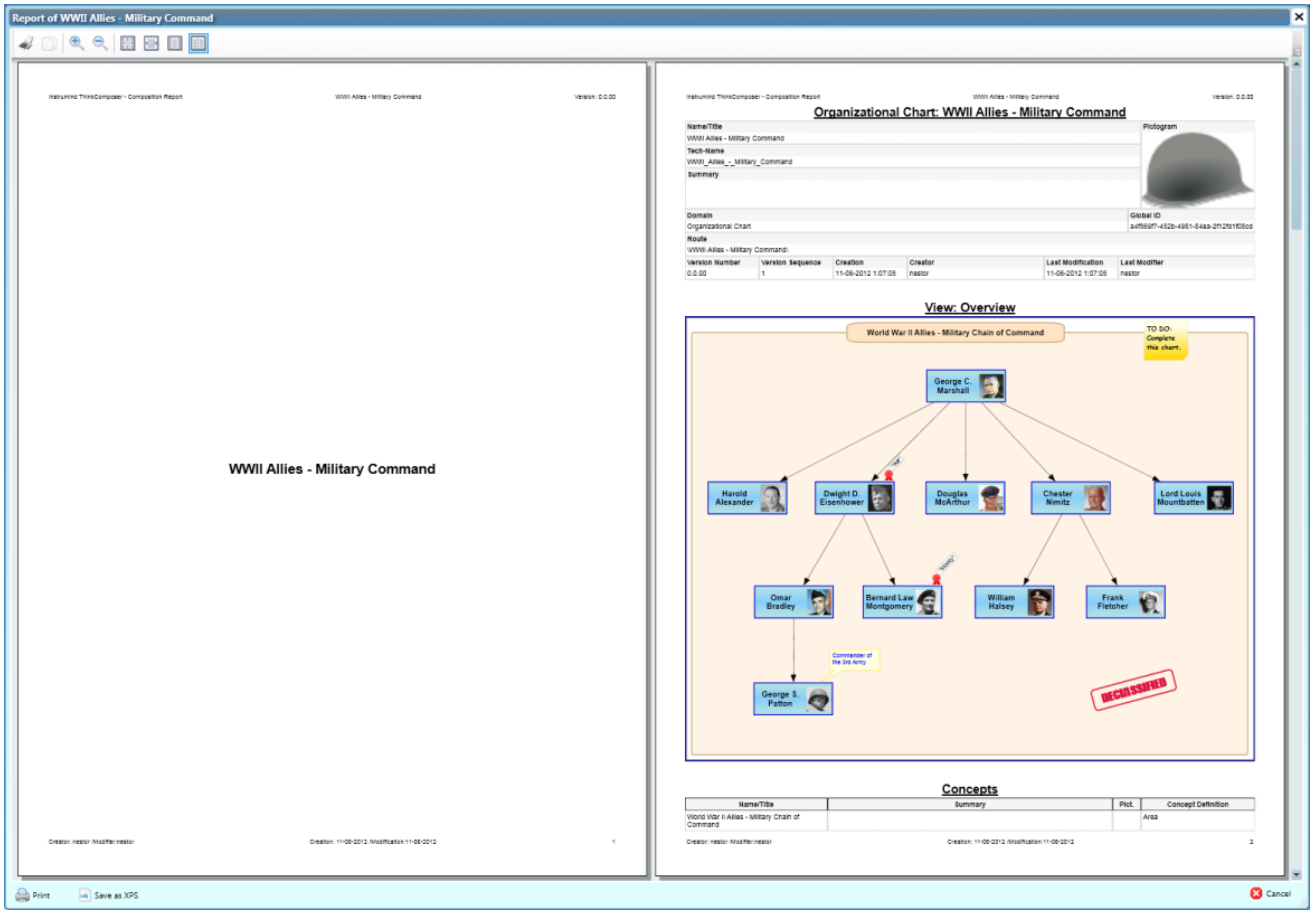

Figure 3.5: Report preview

For scripted report generation, see Reports.

For more controlled text or code output, use Output Templates through `Tools -> Output -> Generation Preview` and `Tools -> Output -> Generate Files...`.

4 Current Features

This chapter integrates the current user-facing features documented in the repository. The detailed technical references remain in the sibling docs, but the workflows below are the practical manual entry points.

Related references:

- Appearance and Layout Tools
- ThinkComposer JSON Interchange
- ThinkComposer Domain JSON Interchange
- Domain Sync
- Output Template Generation
- Command-Line Interface
- Layout Services

4.1 Appearance And Layout Tools

Appearance commands live under:

```
Edit -> Appearance
```

They move, resize, or route visible representations. They do not change concept or relationship meaning.

Command	Use when	Changes
Fit Concept Width to Text	Labels are clipped, too wide, or recently edited.	Selected concept symbol widths.
Route Links with Obstacle Avoidance	Connector lines cross concept symbols.	Connector intermediate points and hidden relationship junctions.
Arrange as Spider Map	One central idea should radiate to related ideas.	Concept positions and routed links.
Arrange as Hierarchy Map	Roots and parents should appear above children/dependencies.	Concept rows, relationship bubbles, and routed links.
Arrange as Flowchart	A process should read left to right.	Flow steps, feedback lanes, relationship bubbles, and routed links.
Arrange as System Map	A system boundary should separate internal components from external actors.	Internal/external positions, Group Region, relationship bubbles, and routed links.

4.1.1 Selection And Undo

- If concept symbols are selected, arrangement commands operate on the selected concepts and visible relationships among them.
- If no concepts are selected, arrangement commands ask before arranging all visible concepts in the active view.

- Link routing operates on selected connectors or relationship visuals; if none are selected, it asks before routing all visible links.
- Each command runs as one undoable ThinkComposer command variation.
- Results are native visual model changes and persist after save, close, and reopen.

4.1.2 Fit Concept Width To Text

This command measures the visible concept title text, applies conservative padding, respects width limits, and preserves symbol height. It also runs when a user double-clicks a selected concept's left or right resize handle.

4.1.3 Route Links With Obstacle Avoidance

The router uses the existing connector route model. It preserves valid hand-routed connectors, uses straight routes when possible, and otherwise tries simple orthogonal candidates.

For simple relationships with a hidden central symbol, routing treats the relationship as one unit. The hidden relationship symbol may become the route junction, allowing practical dogleg behavior without adding a new multi-point route model.

4.1.4 Arrange As Spider Map

Spider Map chooses a root, places it near the center, puts first-level neighbors around it, and places remaining concepts on a second ring. It auto-fits labels first and routes in-scope links after movement.

4.1.5 Arrange As Hierarchy Map

Hierarchy Map interprets relationship direction when available, chooses roots, assigns levels with a guarded breadth-first traversal, places disconnected components side by side, and declutters visible relationship central symbols.

4.1.6 Arrange As Flowchart

Flowchart arranges concepts as left-to-right process steps. Feedback, reverse, and long cross-link relationships are placed into a feedback lane outside the main process band.

4.1.7 Arrange As System Map

System Map detects or uses a selected system/root concept, classifies internal and external concepts, places internals inside a visible Group Region, and places external actors on the left or right of the boundary.

The Group Region is visual only. It does not change semantic containment, composite ownership, or relationship links.

4.2 Composition JSON Interchange

Composition JSON Interchange exports a `.tcom` composition to editable JSON and safely applies edited JSON into an updated package.

Modern `.tcom` packages also use root `/Composition.json` and `/Domain.json` as their authoritative Open/Save payloads. Desktop Composition JSON import/export buttons are deprecated; use package root JSON for native persistence review and the command-line interface for explicit Composition JSON interchange.

Native JSON package persistence preserves supported visual state such as positions, colors, connector paths, grouping complements, shortcut visuals, active/root view identity, and visible detail posters. Binary package parts, when present, are legacy fallback only.

4.2.1 Workflow

1. Open a composition in ThinkComposer.
2. Run `thinkcomposer composition export-json --input <file.tcom> --output <file.json>`.
3. Edit the JSON manually, with tools, or with AI assistance.
4. Run `thinkcomposer composition import-json --input <file.tcom> --json <file.json> --output <updated-file.tcom>`.
5. Review the diagnostics.
6. Open the updated `.tcom` file when the result is correct.

Every supported document starts with:

```
{
  "format": "ThinkComposer.JsonInterchange",
  "formatVersion": 1,
  "application": "ThinkComposer"
}
```

Unknown JSON fields are ignored. The schema is maintained at ../thinkcomposer-json-interchange.schema.json.

4.2.2 Full-State Merge

Full-state import updates matching objects by `id` first, then by `techName`. Omitted objects are never deleted.

By default, missing top-level ideas, relationships, and views are treated as updates to existing objects, not creates. To use a generated full-state-style document to populate a blank composition, opt in explicitly:

```
{
  "importOptions": {
    "useActiveCompositionAsContainer": true,
    "treatMissingFullStateItemsAsCreates": true
  }
}
```

Patch operations remain preferred for AI-authored creation because they make intent, order, and safety easier to inspect.

4.2.3 Patch Operations

Supported patch operations include `update`, `create`, `delete`, and `place` for supported entity types.

Concept creates require a concept definition and a container. Relationship creates require a relationship definition, a container, and valid origin/target links. Place operations create or update visuals in a target view.

Use the root sentinel for generated root-level patches:

```
{
  "importOptions": {
    "useActiveCompositionAsContainer": true
  },
  "operations": [
    {
      "op": "create",
      "entity": "concept",
      "definitionTechName": "Concept",
      "containerTechName": "Active_Composition_Root",
      "set": {
        "name": "New concept",
        "techName": "NewConcept"
      }
    }
```

```
  ]
}
```

4.2.4 Visual Strategy

Large generated models should not create, auto-fit, and auto-route every visual by default. Use top-level `visualStrategy` to describe how much visual materialization is intended.

Modes include:

Mode	Use
`modelOnly`	Create semantic data while suppressing visual placement.
`overviewAndModel`	Create the full model and a capped overview.
`optimizedFullVisual`	Create full visuals but defer expensive fitting/routing/refresh work.
`exactFullVisual`	Preserve exact placement for small curated diagrams.
`auto`	Choose based on configured thresholds.

4.2.5 Relationship Center Placement

ThinkComposer relationships can have visible central symbols. Generated diagrams should usually place relationship centers near their endpoints, not in a distant global label row.

Recommended setting:

```
{
  "importOptions": {
    "relationshipVisualPlacementMode": "endpointCorridor",
    "recomputeSuspiciousRelationshipVisuals": true
  }
}
```

Use `explicit` only when coordinates are intentionally curated.

4.2.6 Shortcuts

Composition JSON exports shortcut visual representations with:

```
{
  "isShortcut": true
}
```

Patch-style placement can request the same behavior with `visual.isShortcut: true`. A shortcut is visual identity, not a duplicate concept.

4.2.7 Intent-Agnostic Import Primitives

The importer does not infer group regions, membership, layout roles, or hidden relationships from source-format names. Use explicit primitives:

- top-level `groups[]`
- concept `visual.role`
- relationship `layoutRole`
- `visual.display`
- `includeInArrangement`
- `includeInRouting`
- `includeInAutoFit`
- per-relationship `relationshipCenterPlacement`

This keeps the importer source-neutral and predictable.

4.2.8 Diagnostics And Safety

Import and export write detailed diagnostics to the application log. Dialogs distinguish:

- source warnings
- import warnings
- skipped operations
- dangerous skipped operations
- notes
- errors

Strict relationship/detail compatibility options can block partial imports before apply. Non-strict workflows still import compatible objects and report invalid relationships or unsupported details.

4.3 Domain JSON Interchange

Domain JSON is the text-safe payload used by modern `.tdom` packages and compatibility merge paths. It is also the merge source for updating an existing composition's embedded domain snapshot.

Modern `.tdom` packages use root `/Domain.json` as their authoritative Open/Save payload. The old Domain JSON import/export desktop controls are deprecated; external edits should patch the authoritative package JSON and then reopen/save the package normally. CLI Domain JSON commands remain available for validation and compatibility workflows.

Supported domain work includes:

- domain metadata updates
- descriptions and TechSpec
- concept definitions
- relationship definitions
- link role definitions
- marker definitions
- table definitions and fields
- visual symbol and connector formats
- report configuration
- base tables
- external languages
- output templates

Domain JSON documents use:

```
{
  "format": "ThinkComposer.DomainJsonInterchange",
  "formatVersion": 1
}
```

The schema is maintained at ../thinkcomposer-domain-json-interchange.schema.json.

4.4 Embedded Domain Updates

Use `Composition -> Domain -> Update Embedded Domain...` when an existing `.tcom` composition should pick up safe additions or updates from a newer `.tdom` source.

The update:

- updates the embedded domain snapshot explicitly

- does not create a live sync link
- does not delete legacy embedded-domain objects by omission
- treats output templates as text during import/update
- prepares imported templates automatically during generation

4.5 Output Template Generation

Output templates generate text files from a composition, concept, or relationship using the active external language.

4.5.1 Preview Scopes

`Tools -> Output -> Generation Preview` works for the active generation scope:

- No selected idea previews the active composition/root scope.
- One selected concept or relationship previews that selected item.
- Multiple selected items preview the first selected item and record the choice in diagnostics.

The preview window includes:

- Rendered Output
- Effective Template
- Resolution

4.5.2 Generation Flow

`Tools -> Output -> Generate Files...` follows this high-level flow:

1. Save the selected external language and target directory.
2. Prepare output templates for the active composition.
3. Lint templates.
4. Abort before writing if blocking preparation issues exist.
5. Rebuild the subtemplate registry deterministically.
6. Render document-root templates through DotLiquid.
7. Suppress fragments/subtemplates as standalone deliverables by default.
8. Post-process and validate XML/JSON-like output when appropriate.
9. Log per-file resolution and a generation summary.

4.5.3 Template Roles

Templates can declare roles:

```
%%:TemplateRole=DocumentRoot
%%:TemplateRole=Fragment
%%:TemplateRole=SubTemplate
%%:TemplateRole=Diagnostic
%%:TemplateRole=NotApplicable
%%:TemplateRole=Disabled
```

`DocumentRoot` templates emit files. Fragment, SubTemplate, Disabled, and NotApplicable templates are not emitted as final files by default.

4.5.4 Linting And Validation

Template preparation checks:

- missing required subtemplates
- duplicate subtemplate names

- obvious recursive injection
- empty template bodies
- XML declaration whitespace risks
- fragment templates that look like full documents
- XML attributes filled directly from expressions that may become blank
- invalid template section parsing

For XML-like or JSON-like outputs, generated text can be post-processed and parsed for validation warnings.

4.5.5 Safe Template Helpers

Current helper filters include:

```
{{ Name | EscapeXmlAttribute }}
{{ Summary | EscapeXmlText }}
{{ TechName | NormalizeTechName }}
{{ Value | DefaultIfEmpty: 'unknown' }}
{{ Info | DetailValue: 'FieldTechName' }}
{{ Value | JsonString }}
```

Use these helpers for XML attributes/elements, JSON strings, fallback text, normalized identifiers, and detail lookups.

4.6 Recommended AI-Assisted Workflow

1. Save or copy the native `.tcom` or `.tdom`.
2. Export JSON when a full-state reference is useful.
3. Ask the assistant to produce patch operations against the schema.
4. Preview import/update in ThinkComposer.
5. Confirm only after reviewing skipped and dangerous skipped counts.
6. Save the native file after successful apply.
7. Re-export JSON when another edit cycle needs a fresh snapshot.

For large generated models, prefer `modelOnly` or `overviewAndModel` visual strategies first, then use manual Appearance tools for final diagram layout.

5 Command-Line Interface

ThinkComposer includes a headless command-line interface for repeatable import, export, package persistence validation, report, and output-generation work. The desktop application remains the primary visual editor. Use the CLI when you want the same operations available from Command Prompt, scripts, build jobs, or other automation.

Modern `.tcom` and `.tdom` files use root JSON payloads as their native persistence source of truth. Manual JSON exports are still exchange artifacts, while PDF, XPS, and generated files are publication/output artifacts.

Related manual topics:

- Overview explains how compositions and domains fit into a normal ThinkComposer workflow.
- Base Model defines compositions, domains, ideas, relationships, output templates, and external languages.
- Application Guide covers installation, working with compositions, reporting, and the desktop commands mirrored by the CLI.
- Current Features explains JSON interchange, Domain JSON, embedded-domain updates, and output template generation.
- Template Language and Composition Information Model describe the template model used by generated output.

Detailed technical references are also available for Composition JSON Interchange, Domain JSON Interchange, Domain Sync, Output Template Generation, and the compact CLI reference.

5.1 When To Use The CLI

Use the CLI for:

- exporting a composition to JSON for compatibility review
- importing reviewed Composition JSON through the compatibility merge path
- exporting a native `.tdom` domain or a composition's embedded domain to JSON for compatibility review
- importing Domain JSON through the compatibility merge path
- updating a composition's embedded domain directly from a native `.tdom`
- inspecting, converting, and validating JSON-authoritative `.tcom` and `.tdom` packages
- generating a standard composition report as PDF or XPS
- generating files from output templates without opening the desktop shell

Use the desktop application when you need to visually edit diagrams, inspect a composition, tune report settings, design output templates, or review import diagnostics interactively.

5.2 Installation And PATH

The installer deploys the command-line files beside the desktop application:

- `ThinkComposer.Cli.exe`
- `thinkcomposer.cmd`
- `thinkcomposer-path.cmd`

- `thinkcomposer-path.ps1`

The installer attempts to add the ThinkComposer install folder to the machine `Path` so `thinkcomposer` works from a new Command Prompt. Open a new Command Prompt after installation and test:

```
thinkcomposer --help
where thinkcomposer
```

If `thinkcomposer` is not recognized, use the installed helper. From the ThinkComposer Start Menu folder, run **Add ThinkComposer to PATH**. You can also open Command Prompt in the ThinkComposer installation folder and run:

```
thinkcomposer --add-to-path
```

The default helper updates the machine `Path` and may prompt for administrator elevation. To update only the current user's `Path`, run:

```
thinkcomposer --add-to-path -User
```

Check or remove the entry with:

```
thinkcomposer --path-status
thinkcomposer --path-status -User
thinkcomposer --remove-from-path
thinkcomposer --remove-from-path -User
```

Open a new Command Prompt after adding or removing a `Path` entry. Existing terminal windows usually keep the old environment.

In PowerShell, if you are already in the installation folder and the command is not on `Path` yet, prefix the shim with `.\`:

```
.\thinkcomposer --add-to-path -User
```

5.3 General Syntax

Use global or command-specific help:

```
thinkcomposer --help
thinkcomposer composition --help
thinkcomposer domain --help
thinkcomposer report --help
thinkcomposer output --help
```

Quote paths that contain spaces:

```
thinkcomposer composition export-json --input "C:\Work Models\Service
 ↪  Map.tcom" --output "C:\Exports\Service Map.json"
```

Exit codes:

Code	Meaning
0	Success.
1	Usage, validation, or expected operation failure.
2	Unexpected exception.

Headless commands initialize ThinkComposer services and drawing resources without showing the WPF shell. Report generation still uses WPF document primitives internally, but no application window is created.

5.4 Import Safety

Imports always require `--output`. The CLI refuses to overwrite the input file unless both conditions are true:

1. `--in-place` is present.
2. `--output` is the same path as `--input`.

Use `--preview-only` to validate the JSON and print the planned import summary without saving any document. A preview still requires `--output` so the command line is the same as the eventual import.

For important work, write imports to a new output file first, open the result in ThinkComposer, and only then decide whether to replace the original.

5.5 Composition JSON

Composition JSON export writes an editable interchange document from a `.tcom` composition:

```
thinkcomposer composition export-json --input "Models\ServiceMap.tcom"
 ↪  --output "Exports\ServiceMap.composition.json"
```

Composition JSON import merges a JSON document back into a composition and writes a `.tcom` output:

```
thinkcomposer composition import-json --input "Models\ServiceMap.tcom"
 ↪  --json "Patches\ServiceMap.patch.json" --output
 ↪  "Models\ServiceMap.updated.tcom"
```

Preview the same operation without saving:

```
thinkcomposer composition import-json --input "Models\ServiceMap.tcom"
 ↪  --json "Patches\ServiceMap.patch.json" --output
 ↪  "Models\ServiceMap.updated.tcom" --preview-only
```

Overwrite the input only when that is intentional:

```
thinkcomposer composition import-json --input "Models\ServiceMap.tcom"
 ↪  --json "Patches\ServiceMap.patch.json" --output
 ↪  "Models\ServiceMap.tcom" --in-place
```

Modern `.tcom` persistence uses root `/Composition.json` and `/Domain.json` as the native source of truth. Use the CLI import/export commands when you need a compatibility merge, preview, or standalone interchange document; patch root package JSON directly for JSON-authoritative persistence edits.

For the JSON model, patch operations, visual strategies, and diagnostics, see Composition JSON Interchange and the detailed Composition JSON Interchange reference.

5.6 Domain JSON

Domain JSON export works with native `.tdom` domain files as a compatibility/interchange command:

```
thinkcomposer domain export-json --input "Domains\ServiceDesign.tdom"
 ↪  --output "Exports\ServiceDesign.domain.json"
```

It can also export the embedded domain snapshot from a `.tcom` composition:

```
thinkcomposer domain export-json --input "Models\ServiceMap.tcom" --output
 ↪  "Exports\ServiceMap.embedded-domain.json"
```

Import Domain JSON into a native domain through the compatibility merge path:

```
thinkcomposer domain import-json --input "Domains\ServiceDesign.tdom"
 ↪  --json "Patches\ServiceDesign.domain.json" --output
 ↪  "Domains\ServiceDesign.updated.tdom"
```

Import Domain JSON into a composition's embedded domain through the compatibility merge path:

```
thinkcomposer domain import-json --input "Models\ServiceMap.tcom" --json
 ↪  "Patches\ServiceDesign.domain.json" --output
 ↪  "Models\ServiceMap.updated.tcom"
```

Update a composition's embedded domain directly from a native `.tdom` source:

```
thinkcomposer domain update-embedded --input "Models\ServiceMap.tcom"
 ↪  --domain "Domains\ServiceDesign.tdom" --output
 ↪  "Models\ServiceMap.updated.tcom"
```

Preview or in-place behavior follows the same safety rules as Composition JSON import:

```
thinkcomposer domain import-json --input "Domains\ServiceDesign.tdom"
 ↪  --json "Patches\ServiceDesign.domain.json" --output
 ↪  "Domains\ServiceDesign.updated.tdom" --preview-only
thinkcomposer domain import-json --input "Domains\ServiceDesign.tdom"
 ↪  --json "Patches\ServiceDesign.domain.json" --output
 ↪  "Domains\ServiceDesign.tdom" --in-place
thinkcomposer domain update-embedded --input "Models\ServiceMap.tcom"
 ↪  --domain "Domains\ServiceDesign.tdom" --output
 ↪  "Models\ServiceMap.updated.tcom" --preview-only
```

Modern `.tdom` persistence uses root `/Domain.json` as the native source of truth. Use CLI domain import/export when you need a compatibility merge, preview, or standalone interchange document; patch root package JSON directly for JSON-authoritative persistence edits.

For domain concepts, see Domains, Output Templates, and External Languages. For merge behavior, see Domain JSON Interchange, Embedded Domain Updates, and the detailed Domain JSON Interchange reference.

5.7 Package Persistence

Inspect a native package:

```
thinkcomposer package inspect --input "Models\ServiceMap.tcom"
```

Convert a legacy binary-backed composition or domain to the JSON-authoritative package format:

```
thinkcomposer composition convert-json-persistence --input
 ↪  "Models\LegacyMap.tcom" --output
 ↪  "Models\LegacyMap.json-persistence.tcom"
thinkcomposer domain convert-json-persistence --input
 ↪  "Domains\LegacyDomain.tdom" --output
 ↪  "Domains\LegacyDomain.json-persistence.tdom"
```

Validate normal JSON-first load/save behavior:

```
thinkcomposer composition validate-json-persistence --input
 ↪  "Models\ServiceMap.tcom" --output-dir "Validation\ServiceMap"
thinkcomposer domain validate-json-persistence --input
 ↪  "Domains\ServiceDesign.tdom" --output-dir "Validation\ServiceDesign"
```

The validation commands save a modern package, reopen it through normal load, fail if binary fallback was used, save again, and compare canonical root JSON payloads.

5.8 Reports

Generate a standard composition report as PDF:

```
thinkcomposer report pdf --input "Models\ServiceMap.tcom" --output
 ↪  "Reports\ServiceMap.pdf"
```

You can also write the intermediate XPS format:

```
thinkcomposer report pdf --input "Models\ServiceMap.tcom" --output
 ↪  "Reports\ServiceMap.xps"
```

This uses the existing ThinkComposer standard report workflow and the saved or default report configuration. If the report layout or included sections are not what you expect, open the composition in the desktop application and review the reporting workflow described in Reporting.

5.9 Output Generation

Generate files from output templates:

```
thinkcomposer output generate --input "Models\ServiceMap.tcom"
 ↪  --output-dir "Generated\ServiceMap" --language "Xml"
```

The `--language` value is the external language TechName from the composition's embedded domain. If you are not sure which language names are available, inspect root `/Domain.json`, run `thinkcomposer domain export-json` as a compatibility export, or review the domain in the desktop application.

Common options:

Option	Meaning
`--relationships`	Also emit standalone relationship files when templates allow them.
`--composition-root-dir`	Create a composition-root directory inside the target output directory.
`--use-tech-names`	Use TechNames as programming identifiers during generation.
`--exclude <idea-id>`	Exclude an idea by GlobalId or TechName. Repeat the option to exclude more than one idea.

Example with options:

```
thinkcomposer output generate --input "Models\ServiceMap.tcom"
 ↪  --output-dir "Generated\ServiceMap" --language "Xml" --relationships
 ↪  --composition-root-dir --use-tech-names --exclude "LegacyNode"
```

Output generation uses the same template preparation, linting, subtemplate registration, and post-processing path described in Output Template Generation. Template syntax is documented in Template Language, and the data exposed to templates is summarized in Composition Information Model.

5.10 Troubleshooting

If `thinkcomposer` is not recognized, open a new Command Prompt first. If it is still unavailable, run **Add ThinkComposer to PATH** from the Start Menu or run `thinkcomposer --add-to-path` from the installation folder.

If a new terminal still resolves an old install, run:

```
where thinkcomposer
```

Then remove the stale folder from `Path` or run `thinkcomposer --remove-from-path` from the old installation folder before adding the current one.

If import fails because JSON is invalid, export a fresh JSON document from the same source file and compare the format header, `formatVersion`, identifiers, and patch operations. Use `--preview-only` before saving.

If output generation reports an unknown language TechName, inspect root `/Domain.json` from the same `.tcom` or `.tdom`, or run `thinkcomposer domain export-json` as a compatibility export, and confirm the intended external language TechName.

If report generation writes an empty or unexpected report, open the composition in the desktop application and verify that the composition opens correctly and that the report configuration is suitable for that document.

6 Appendix A: Template Language

ThinkComposer Output Templates use Liquid markup plus ThinkComposer-specific control markup. Templates reference properties from the Composition Information Model and merge those values with template text to generate files.

The original Liquid language reference is available at:

https://github.com/Shopify/liquid/wiki/Liquid-for-Designers

6.1 Control Markup

Control markup tells ThinkComposer what to do with generated text. It is prefixed with `%%:`.

Control	Description
`%%:FileName=<Template-Text>`	Sets the generated file name. It must be the first line and consume the whole line. If omitted, generated files use the idea TechName and `.txt`.
`%%:[ExtensionPlace]`	Marks where text from templates that extend a base template should be inserted. If omitted, extension text is appended.
`%%:SubTemplate=<Identifier>`	Declares a reusable subtemplate from the next line until another subtemplate declaration or the end of the template.
`%%:TemplateRole=<Role>`	Declares the modern template role, such as `DocumentRoot`, `Fragment`, `SubTemplate`, `Diagnostic`, `NotApplicable`, or `Disabled`.
`%%:outputPostProcess.<option>=<value>`	Controls output cleanup such as trimming leading whitespace or normalizing line endings.
`%%:outputValidation=<Mode>`	Requests validation such as XML well-formedness.

Example file-name control:

```
%%:FileName=Idea-{{ TechName }}.txt
[MyDocStart]
My text
[MyDocEnd]
```

Example extension place:

```
%%:FileName=Idea-{{ TechName }}.txt
[MyDocStart]
Base text
%%:[ExtensionPlace]
[MyDocEnd]
```

Example subtemplate:

```
%%:FileName=Composition-{{ TechName }}.txt
[MyDocStart]
Root: {{ Name }}
{%- inject 'TreeNodeReader' with This -%}
[MyDocEnd]

%%:SubTemplate=TreeNodeReader
[NodeStart]
Node-Name: {{ Name }}
{% assign Depth = @@InjectionDepth %}
Nested-Level: {{ Depth }}
{%- for Idea in CompositeIdeas -%}
{%- inject 'TreeNodeReader' with Idea -%}
{%- endfor -%}
[NodeEnd]
```

6.2 Output Markup

Output markup writes evaluated text to the generated output. It is enclosed between `{{` and `}}`.

An expression may contain:

- a property name
- an indexed lookup
- a literal value
- filters applied with `|`
- whitespace trimming markers using `−`

Examples:

```
Title: {{ Name }}
{{ Name }} is a {{ Summary | Upcase }}.
"{{ Name }}" has {{ Name | Size }} characters.
```

Whitespace trimming:

```
Name: {{- Name }}
Summary:
{{ Summary -}}
;
```

6.3 Filters

Filters take input from the left side and return transformed output. Names are case-sensitive.

Filter	Description
`Size`	Gets the size of an array, string, or collection. Also usable as a property.
`Any`	Indicates whether a collection or string has content. Also usable as a property.
`AsChar`	Converts `tab`, `newline`, or a UTF-16 code to a character.
`ToBase64`	Gets binary content as Base64 text.
`ToUnformattedText`	Converts rich text such as `Description` to unformatted text.

Filter	Description
ToPlainText	Converts an arbitrary object such as detail content to simple text.
Capitalize	Capitalizes words in the input sentence.
Downcase	Converts a string to lowercase.
Upcase	Converts a string to uppercase.
First	Gets the first element of an array or collection.
Last	Gets the last element of an array or collection.
Join	Joins array elements with a separator.
Sort	Sorts array elements.
Map	Maps a collection to a property.
Escape	Escapes a string.
EscapeOnce	Escapes HTML without affecting existing escaped entities.
StripHtml	Removes HTML tags.
StripNewlines	Removes newline characters.
NewlineToBr	Replaces newlines with HTML line breaks.
Replace	Replaces each occurrence.
ReplaceFirst	Replaces the first occurrence.
Remove	Removes each occurrence.
RemoveFirst	Removes the first occurrence.
Truncate	Truncates a string to a character count.
Truncatewords	Truncates a string to a word count.
Prepend	Prepends a string.
Append	Appends a string.
Minus	Numeric subtraction.
Plus	Numeric addition, or string concatenation when values are strings.
Times	Numeric multiplication.
DividedBy	Numeric division.
Split	Splits a string on a matching pattern.
Modulo	Numeric remainder.
Get	Gets a property from a source object by TechName.
GetIdeasDefinedAs	Filters ideas by Idea Definition TechName.
GetElements	Filters identifiable elements by TechName.
GetLinksByVariant	Filters role-based links by role variant TechName.
SelectMany	Flattens nested collections from a named collection property.

Examples:

```
Items = {{ Source | Size }}
{{ 'da vinci' | Capitalize }}
{{ 'foofoo' | replace:'foo','bar' }}
{{ 5 | times:4 }}
{% assign Arachnids = Animals | GetIdeasDefinedAs:'Spider;Scorpion' %}
{% assign AllSolarSystemMoons = Planets | SelectMany:'Moons' %}
```

6.4 Safe Modern Filters

Current generation also supports safer helpers for XML, JSON, identifiers, and detail lookup:

Filter	Use
`EscapeXmlAttribute`	Escape text for XML attributes.
`EscapeXmlText`	Escape text for XML element content.
`NormalizeTechName`	Normalize text into a technical identifier.
`DefaultIfEmpty`	Provide fallback text when a value is blank.
`DetailValue`	Read a simple field value from a detail/table-like object.
`JsonString`	Escape text for JSON string values.

Example:

```
<device id="{{ TechName | NormalizeTechName | EscapeXmlAttribute }}"
        name="{{ Name | DefaultIfEmpty: 'Unnamed' | EscapeXmlAttribute }}"
 ↪  />
```

6.5 Tag Markup

Tag markup declares processing instructions. Tags are enclosed between `{%` and `%}` and do not directly emit text unless the tag body emits text.

Tag	Description
`assign`	Assigns a value to a variable.
`capture`	Captures generated text into a variable.
`case`	Standard `case` / `when` / `else` block.
`comment`	Comments out a block.
`for`	Iterates through a collection.
`if`	Conditional block.
`inject`	Inserts a subtemplate with a new information context.
`raw`	Temporarily disables tag processing.
`unless`	Inverse conditional block.

Example `for` block:

```
{% for Detail in Details %}
This is a detail of kind {{ Detail.Kind.Name }}
{% if forloop.last %}
This is the last element.
{% endif %}
{% endfor %}
```

Example `if` block:

```
{% if Details.Size < 1 %}
empty
{% else %}
There are {{ Details.Size }} details.
{% endif %}
```

Example `inject`:

```
{% for Child in CompositeIdeas %}
```

```
{% inject 'ChildrenTemplate' with Child %}
{% endfor %}

{% inject 'MyTemplate' with Data keepindent %}
{% inject 'MyTemplate' with Remarks noindent %}
```

6.6 Operators And Escaping

Logical operators include:

- `==`
- `!=`
- `and`
- `or`

Collection operators include:

- `contains`
- `empty`

Separate operators and values with spaces. For example, `a == b` is valid, while `a==b` should be avoided.

Backslashes in function parameters must be escaped by writing them twice:

```
{{ TechName | replace:'\\','.' }}
```

7 Appendix B: Composition Information Model

This appendix summarizes the information model exposed to Output Templates. Templates use this model to read composition, idea, domain, detail, table, and visual information.

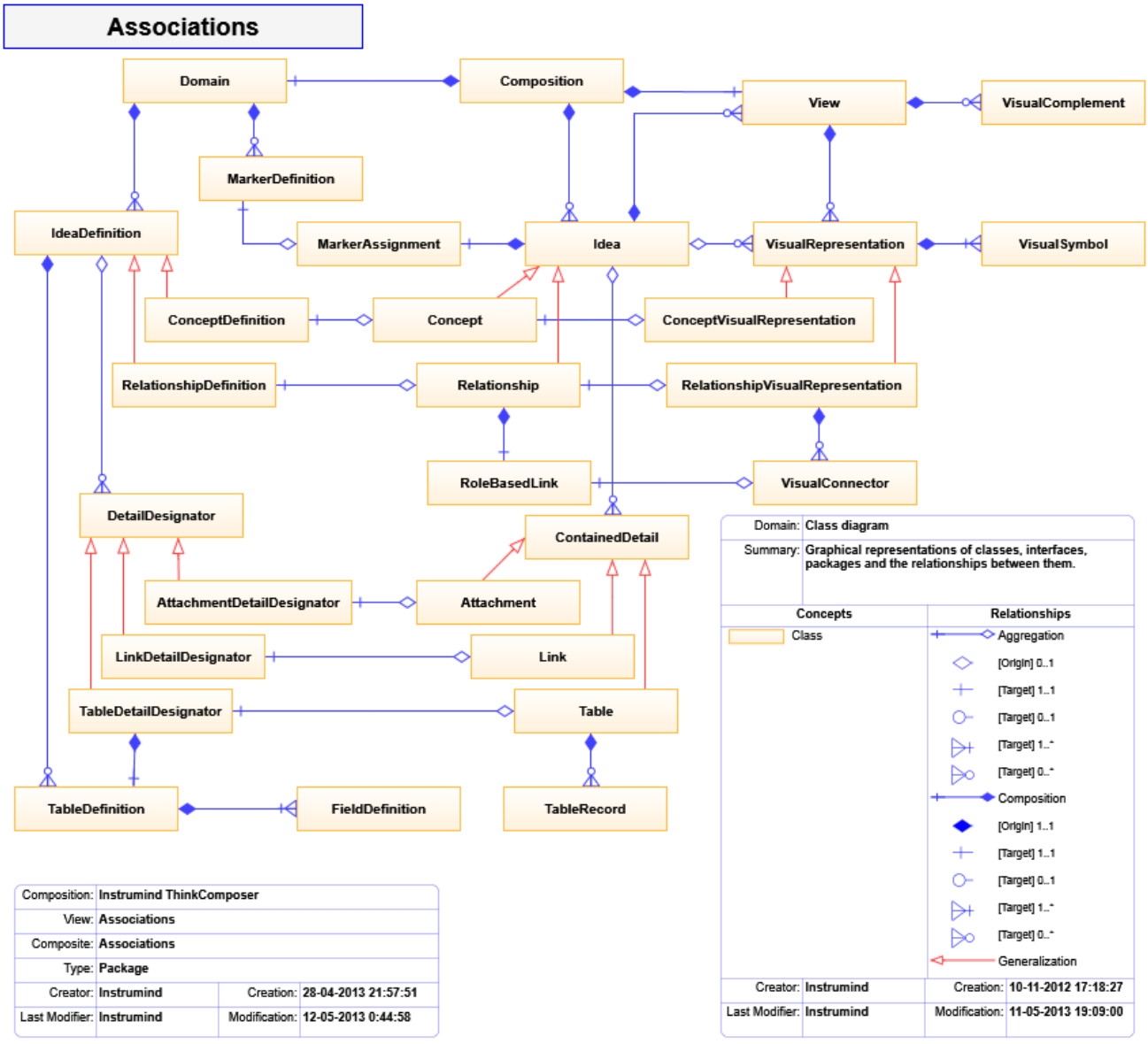

Figure 7.1: Composition information model associations

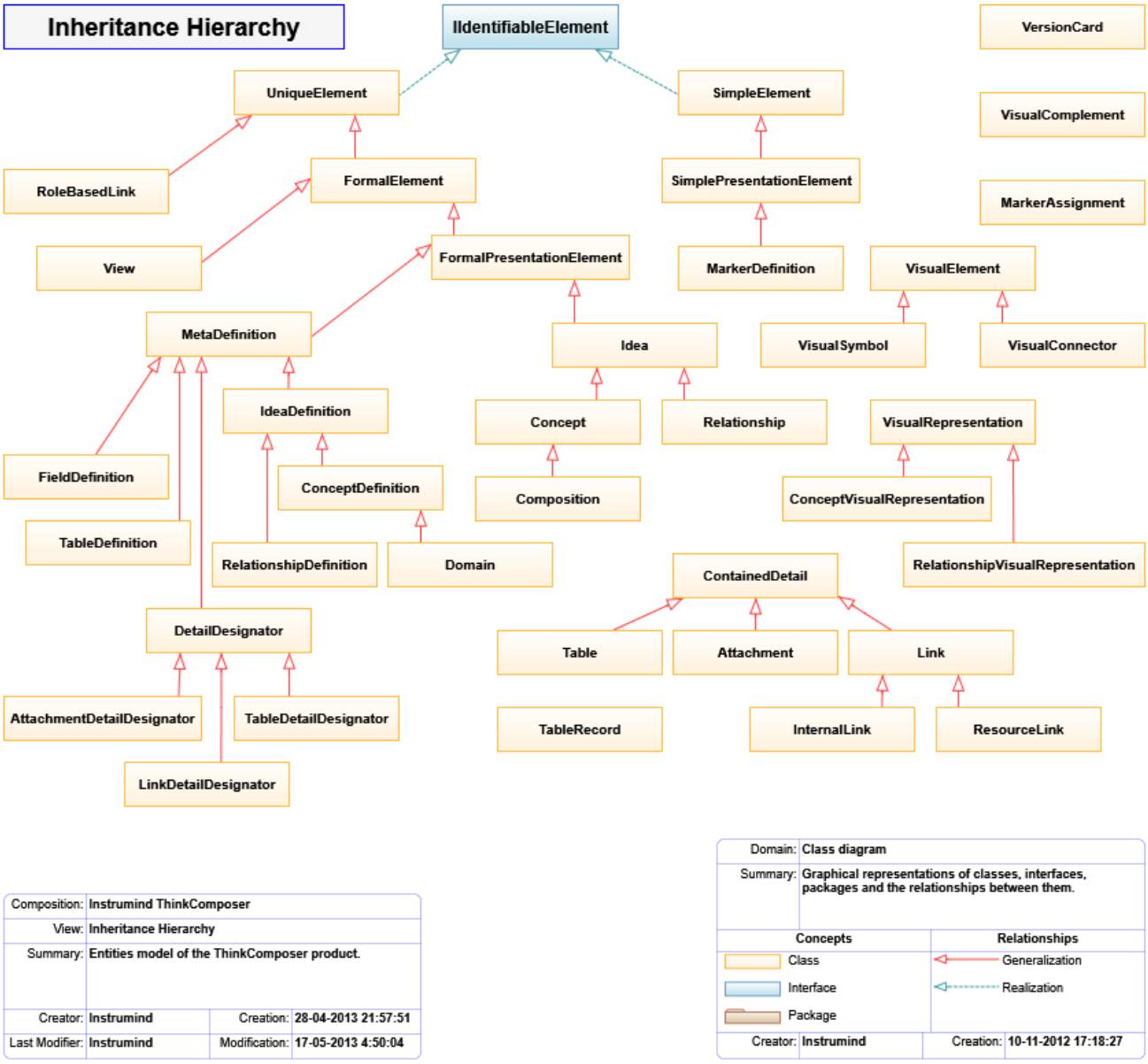

Figure 7.2: Composition information model inheritance

7.1 Special Access Patterns

Use these access patterns when a template needs dynamic fields, details, table rows, or domain base tables.

Custom Fields

- Access: `_['<CustomFieldTechName>']`
- Example: `{{ _['Alias'] }}`

Details

- Access: `This['<DetailTechName>']`
- Example: `{{ This['MyTable'].Records.Size }}`

Table Records

- Access: `Record.FieldTechName` or `Record['FieldTechName']`
- Example: `{{ Person.Name }}; {{ Person['Age'] }}`

Domain Base Tables

- Access: `<Domain>.BaseContentRoot['<BaseTableTechName>']`
- Example: `{{ OwnerComposition.CompositionDefinitor.BaseContentRoot['States'] }}`

Example:

```
The {{ This['People'].Records.Size }} persons are:
{% for Person in This['People'].Records %}
Name: {{ Person.Name }}; Age: {{ Person['Age'] }}
{% endfor %}
```

7.2 Core Classes

The model classes below are written as compact entries instead of a wide table so long class names and ancestors remain readable in the generated PDF.

`Attachment`

- Ancestor: `ContainedDetail`
- Summary: Embedded object from an external source, such as an image or file.

`AttachmentDetailDesignator`

- Ancestor: `DetailDesignator`
- Summary: Associates an attachment definition to an idea.

`Composition`

- Ancestor: `Concept`
- Summary: Semantic, informational, and visual set of ideas expressing knowledge about a subject.

`Concept`

- Ancestor: `Idea`
- Summary: Concrete object that can be associated to others through relationships.

`ConceptDefinition`

- Ancestor: `IdeaDefinition`

- Summary: Definition of a concept type.

ConceptVisualRepresentation

- Ancestor: VisualRepresentation
- Summary: Visual representation of a concept.

ContainedDetail

- Ancestor: none
- Summary: Object stored as detail for an idea.

DetailDesignator

- Ancestor: MetaDefinition
- Summary: Base ancestor for detailed-data designators.

Domain

- Ancestor: ConceptDefinition
- Summary: Metamodel for graph, visual, and information structures.

FieldDefinition

- Ancestor: MetaDefinition
- Summary: Defines a table field.

FormalElement

- Ancestor: UniqueElement
- Summary: Standard object with name, TechName, summary, TechSpec, rich description, and version.

FormalPresentationElement

- Ancestor: FormalElement
- Summary: Standard object with visual representation.

Idea

- Ancestor: FormalPresentationElement
- Summary: Base composition element from which concepts and relationships descend.

IdeaDefinition

- Ancestor: MetaDefinition
- Summary: Shared ancestor for concept and relationship definitions.

InternalLink

- Ancestor: Link
- Summary: References an internal property.

Link

- Ancestor: ContainedDetail
- Summary: References an external or internal object.

LinkDetailDesignator

- Ancestor: DetailDesignator

- Summary: Associates a link definition to an idea.

LinkRoleDefinition

- Ancestor: `MetaDefinition`
- Summary: Defines a relationship link role.

MarkerAssignment

- Ancestor: none
- Summary: Assignment of a marker to an idea, optionally with descriptor.

MetaDefinition

- Ancestor: `FormalPresentationElement`
- Summary: Definition-level object used to create schema objects.

ModelDefinition

- Ancestor: none
- Summary: Base classifier/member definition.

Relationship

- Ancestor: `Idea`
- Summary: Association between ideas through role-based links.

RelationshipDefinition

- Ancestor: `IdeaDefinition`
- Summary: Definition of a relationship type.

RelationshipVisualRepresentation

- Ancestor: `VisualRepresentation`
- Summary: Visual representation of a relationship.

ResourceLink

- Ancestor: `Link`
- Summary: References a file, folder, web address, or other external resource.

RoleBasedLink

- Ancestor: `UniqueElement`
- Summary: Links a relationship to a related idea through a role definition.

SimpleElement

- Ancestor: none
- Summary: Basic object with name, TechName, summary, and TechSpec.

SimplePresentationElement

- Ancestor: `SimpleElement`
- Summary: Simple object with a visual representation.

Table

- Ancestor: `ContainedDetail`

- Summary: Structured detail containing records.

TableDefinition

- Ancestor: `MetaDefinition`
- Summary: Defines a table structure.

TableDetailDesignator

- Ancestor: `DetailDesignator`
- Summary: Associates table structures to an idea, definition, or field.

TableRecord

- Ancestor: none
- Summary: One structured record inside a table.

UniqueElement

- Ancestor: none
- Summary: Object with a global unique identifier.

VersionCard

- Ancestor: none
- Summary: Version-control information for versionable objects.

View

- Ancestor: `FormalElement`
- Summary: Visual representation of the composite content of a composition or idea.

VisualComplement

- Ancestor: `VisualObject`
- Summary: Visual object exposing attached information such as notes, callouts, legends, and info-cards.

VisualConnector

- Ancestor: `VisualElement`
- Summary: Visual connection between symbols.

VisualElement

- Ancestor: `VisualObject`
- Summary: Base ancestor for visual representators such as symbols and connectors.

VisualRepresentation

- Ancestor: `UniqueElement`
- Summary: Groups visual elements that represent an idea in a view.

VisualSymbol

- Ancestor: `VisualElement`
- Summary: Base ancestor for visual symbols.

7.3 Common Properties

The following property groups use one entry per property to keep long generic types readable in the PDF.

7.3.1 FormalElement

Description

- Type: `String`
- Summary: Rich detailed text.

Name

- Type: `String`
- Summary: User-facing title.

NameCaption

- Type: `String`
- Summary: Single-line display name.

Summary

- Type: `String`
- Summary: Short summary.

TechName

- Type: `String`
- Summary: Stable technical identifier.

TechSpec

- Type: `String`
- Summary: Technical text for scripts, templates, formulas, or generated output.

Version

- Type: `VersionCard`
- Summary: Version metadata.

7.3.2 Idea

—

- Type: `TableRecord`
- Summary: Alias of the custom-fields record.

AssociatingLinks

- Type: `EditableList<RoleBasedLink>`
- Summary: Links associating this idea to relationships.

CompositeIdeas

- Type: `EditableList<Idea>`
- Summary: Child ideas when this idea is composite.

CompositeViews

- Type: `EditableList<View>`
- Summary: Views for contained child ideas.

`Details`

- Type: `EditableList<ContainedDetail>`
- Summary: Contained details.

`IncomingLinks`

- Type: `IEnumerable<RoleBasedLink>`
- Summary: Links targeting this idea.

`OutgoingLinks`

- Type: `IEnumerable<RoleBasedLink>`
- Summary: Links originating from this idea.

`OwnerComposition`

- Type: `Composition`
- Summary: Owning composition.

`OwnerContainer`

- Type: `Idea`
- Summary: Owning composite idea.

`RelatedFrom`

- Type: `IEnumerable<Idea>`
- Summary: Ideas pointing to this idea.

`RelatingTo`

- Type: `IEnumerable<Idea>`
- Summary: Ideas pointed to by this idea.

`This`

- Type: `Idea`
- Summary: Self reference supporting detail indexer access.

`VisualRepresentators`

- Type: `EditableList<VisualRepresentation>`
- Summary: Visual representations of this idea.

7.3.3 Composition

`ActiveView`

- Type: `View`
- Summary: Active view.

`CompositionDefinitor`

- Type: `Domain`
- Summary: Domain definition used by the composition.

RootView

- Type: `View`
- Summary: Initial central view.

UsedDomains

- Type: `EditableList<Domain>`
- Summary: Domains used by the composition.

ViewsPrefix

- Type: `String`
- Summary: Prefix for related view names.

7.3.4 Domain

BaseContentRoot

- Type: `Concept`
- Summary: Root for predefined base content such as base tables.

DefaultTableDef

- Type: `TableDefinition`
- Summary: Default table structure.

IdeaClusters

- Type: `EditableList<SimplePresentationElement>`
- Summary: Palette clusters for idea definitions.

LinkRoleVariants

- Type: `EditableList<SimplePresentationElement>`
- Summary: Predefined role variants such as multiplicities.

MarkerClusters

- Type: `EditableList<SimplePresentationElement>`
- Summary: Palette clusters for marker definitions.

OwnerComposition

- Type: `Composition`
- Summary: Composition owning this domain instance.

ViewBackgroundImage

- Type: `ImageSource`
- Summary: Initial background for views.

7.3.5 IdeaDefinition

CanAutomaticallyCreateGroupedConcepts

- Type: `Boolean`
- Summary: Allows automatic grouped concept creation.

CanAutomaticallyCreateRelatedConcepts

- Type: `Boolean`
- Summary: Allows automatic related concept creation.

CanGroupIntersectingObjects

- Type: `Boolean`
- Summary: Allows grouping objects intersecting a symbol or group region.

CompositeContentDomain

- Type: `Domain`
- Summary: Domain governing composite content.

ConceptDefinitions

- Type: `EditableList<ConceptDefinition>`
- Summary: Child concept definitions.

CustomFieldsTableDef

- Type: `TableDefinition`
- Summary: Custom field structure.

DetailDesignators

- Type: `EditableList<DetailDesignator>`
- Summary: Declared detail designators.

HasGroupLine

- Type: `Boolean`
- Summary: Creates ideas with an appended Group Line.

HasGroupRegion

- Type: `Boolean`
- Summary: Creates ideas with an appended Group Region.

IsComposable

- Type: `Boolean`
- Summary: Allows ideas of this definition to contain others.

IsVersionable

- Type: `Boolean`
- Summary: Enables version metadata.

RelationshipDefinitions

- Type: `EditableList<RelationshipDefinition>`
- Summary: Child relationship definitions.

RepresentativeShape

- Type: `String`
- Summary: Shape used for visual symbols.

TableDefinitions

- Type: `EditableList<TableDefinition>`
- Summary: Declared table structures.

7.3.6 Relationship

DescriptiveCaption

- Type: `String`
- Summary: Short text describing relationship links.

IsAutoReference

- Type: `Boolean`
- Summary: Indicates a relationship can link an idea to itself.

IsAutoReferenceExclusive

- Type: `Boolean`
- Summary: Indicates all links point from/to the same idea.

Links

- Type: `EditableList<RoleBasedLink>`
- Summary: Implemented links.

OriginIdeas

- Type: `IEnumerable<Idea>`
- Summary: Source/participant ideas.

OriginLinks

- Type: `IEnumerable<RoleBasedLink>`
- Summary: Source/participant links.

RelationshipDefinitor

- Type: `Assignment<RelationshipDefinition>`
- Summary: Relationship definition assignment.

TargetIdeas

- Type: `IEnumerable<Idea>`
- Summary: Target ideas.

TargetLinks

- Type: `IEnumerable<RoleBasedLink>`
- Summary: Target links.

7.3.7 RelationshipDefinition

AncestorRelationshipDef

- Type: `RelationshipDefinition`
- Summary: Ancestor relationship definition.

HideCentralSymbolWhenSimple

- Type: `Boolean`
- Summary: Hides the central symbol for simple relationships.

IsDirectional

- Type: `Boolean`
- Summary: Indicates source-to-target semantics.

IsSimple

- Type: `Boolean`
- Summary: Allows one source and one target link.

OriginOrParticipantLinkRoleDef

- Type: `LinkRoleDefinition`
- Summary: Origin or participant role definition.

ShowNameIfHidingCentralSymbol

- Type: `Boolean`
- Summary: Shows the name when central symbol is hidden.

TargetLinkRoleDef

- Type: `LinkRoleDefinition`
- Summary: Target role definition.

7.3.8 Table

Count

- Type: `Int32`
- Summary: Number of records.

Definition

- Type: `TableDefinition`
- Summary: Table structure definition.

Records

- Type: `EditableList<TableRecord>`
- Summary: Records in the table.

RecordsLabel

- Type: `String`
- Summary: Text representation of the first records.

7.3.9 View

BackgroundImage

- Type: `ImageSource`
- Summary: View background image.

GridSize

- Type: `Double`
- Summary: Grid size from 2 to 20 pixels.

GridUsesLines

- Type: `Boolean`
- Summary: Uses line grid instead of points.

OwnerCompositeContainer

- Type: `Idea`
- Summary: Composite idea owning this view.

PageDisplayScale

- Type: `Int32`
- Summary: View page scale percentage.

ShowConceptDefinitionLabels

- Type: `Boolean`
- Summary: Shows concept definition labels.

ShowContextGrid

- Type: `Boolean`
- Summary: Shows the grid.

ShowIndicators

- Type: `Boolean`
- Summary: Shows indicators over ideas.

ShowMarkers

- Type: `Boolean`
- Summary: Shows markers.

SnapToGrid

- Type: `Boolean`
- Summary: Aligns object positioning to grid points.

ViewSize

- Type: `Size`
- Summary: View dimensions.

7.3.10 VisualRepresentation

DisplayingView

- Type: `View`
- Summary: View showing this representation.

IsShortcut

- Type: `Boolean`

- Summary: Indicates the visual points to an idea outside the current container.

MainSymbol

- Type: `VisualSymbol`
- Summary: Primary symbol of the representation.

RepresentedIdea

- Type: `Idea`
- Summary: Represented idea.

7.3.11 VisualSymbol

AreDetailsShown

- Type: `Boolean`
- Summary: Indicates whether details are visible.

BaseArea

- Type: `Rect`
- Summary: Symbol heading rectangle.

BaseCenter

- Type: `Point`
- Summary: Symbol center.

BaseHeight

- Type: `Double`
- Summary: Symbol body height.

BaseLeft

- Type: `Double`
- Summary: Symbol body left position.

BaseTop

- Type: `Double`
- Summary: Symbol body top position.

BaseWidth

- Type: `Double`
- Summary: Symbol body width.

Complements

- Type: `EditableList<VisualComplement>`
- Summary: Attached complements such as callouts.

DetailsArea

- Type: `Rect`
- Summary: Details poster area.

ShowCompositeContentAsDetails

- Type: `Boolean`
- Summary: Shows composite content in the details area.

`TotalArea`

- Type: `Rect`
- Summary: Symbol plus details poster area.

7.4 Generic Collection Types

`EditableList<ItemType>`

- Summary: Ordered collection.
- Relevant property: `Count`.

`EditableDictionary<KeyType, ValueType>`

- Summary: Key-value collection.
- Relevant property: `Count`.

`Assignment<KeyType, ValueType>`

- Summary: References a local writable object or an external read-only object.
- Relevant properties: `IsLocal`, `Value`.

`Ownership<GlobalType, LocalType>`

- Summary: References global/shared or local/exclusive ownership.
- Relevant properties: `IsGlobal`, `Owner`.

`StoreBox<ContentType>`

- Summary: Stores content that requires conversion to and from disk format.
- Relevant property: `Value`.

`TableRecord` also exposes dynamic properties based on the Field Definitions created for its Table Definition. Use the special access patterns above when field names are easier to resolve by TechName.